

22203056_Masud_Parvej.pdf

 International University of Business Agriculture and Technology

Document Details

Submission ID

trn:oid:::3618:138770323

Submission Date

May 13, 2026, 12:17 PM GMT+6

Download Date

May 13, 2026, 12:33 PM GMT+6

File Name

22203056_Masud_Parvej.pdf

File Size

3.8 MB

104 Pages

12,522 Words

83,009 Characters

70% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.



126 AI-generated only 66%

Likely AI-generated text from a large-language model.



5 AI-generated text that was AI-paraphrased 3%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Coaching Center Management System

Md. Masud Parvej

ID# 22203056

A Practicum in the Partial Fulfillment of the Requirements
for the Award of Bachelor of Computer Science and Engineering (BCSE)



Department of Computer Science and Engineering
College of Engineering and Technology
IUBAT—International University of Business Agriculture and Technology

Spring 2025

Coaching Center Management System

Md. Masud Parvej

A Practicum in the Partial Fulfillment of the Requirements for the Award of Bachelor of
Computer Science and Engineering (BCSE)

The practicum has been examined and approved,

Prof. Dr. Utpal Kanti Das
Chairman

Shahinur Alam
Co-supervisor, Coordinator and Assistant Professor

Dipta Mohon Das
Lecturer

Department of Computer Science and Engineering
College of Engineering and Technology
IUBAT—International University of Business Agriculture and Technology

Spring 2025

Letter of Transmittal

12 May 2026

The Chair

Practicum Defense Committee

Department of Computer Science and Engineering

IUBAT—International University of Business Agriculture and Technology

4 Embankment Drive Road, Sector 10, Uttara Model Town

Dhaka 1230, Bangladesh.

Subject: Letter of Transmittal.

Dear Sir,

Sincerely, My name is Md. Masud Parvej, and I am pleased to submit my practicum paper titled "Coaching Center Management System," prepared as part of the requirements for completing my Bachelor of Computer Science and Engineering at IUBAT. This project provided me with a valuable opportunity to apply my theoretical knowledge in a practical setting, specifically utilizing the MERN stack and modern web development practices.

I am more than willing to offer any additional explanations or details you may require. Thank you for the opportunity to work on this report and for your guidance throughout this process.

Yours sincerely,

Md. Masud Parvej

ID# 22203056

Organization's Certificate

Student's Declaration

I am Md. Masud Parvej, bearing ID#22203056, a student of the Bachelor of Computer Science and Engineering (BCSE) program at the College of Engineering and Technology (CEAT), International University of Business Agriculture and Technology (IUBAT). I hereby declare that this report, titled "Coaching Center Management System," has been prepared to fulfill the requirements of the internship CSC 490, Practicum, as well as a partial requirement for the BCSE degree.

I am the sole author of this report and the project on the "Coaching Center Management System." The project's modules and processes were developed based on extensive research conducted online and through hands-on technical implementation.

This report has not been used, awarded or presented for any other purpose.

Md. Masud Parvej

ID# 22203056

Supervisor's Certification

This is to certify that the practicum report titled “Coaching Center Management System,” has been executed by Md. Masud Parvej, bearing ID#22203056, of IUBAT– International University of Business Agriculture and Technology, as a partial fulfillment of the practicum defense course requirements.

The report, which was created with my guidance, serves as a comprehensive summary of the successfully completed task. To the best of my knowledge and according to his assertion, no portions of this report have been submitted elsewhere for any degree, diploma, or certification.

He is now ready to submit the report. I wish him success in all his future endeavors.

Dipta Mohon Das

Lecturer

Department of Computer Science and Engineering

IUBAT—International University of Business Agriculture and Technology

Abstract

The Coaching Center Management System (CCMS) was developed as a comprehensive digital solution to address the increasing administrative complexities and fragmented communication workflows prevalent in traditional educational coaching centers. Built using the MERN stack comprising MongoDB, Express.js, React, and Node.js and styled with the modern Tailwind CSS framework, the system provides a centralized platform that seamlessly integrates the needs of administrators, educators and students. The project focuses on transforming manual record-keeping into an automated, real-time ecosystem where administrators can oversee institutional growth, verify payment statuses, and manage user accounts; teachers can efficiently organize course materials, schedule batches, and track student assignments and students can enjoy a transparent learning journey with personalized dashboards to monitor their enrollment status and academic resources. Developed over a twelve-week period through an incremental process model, the system emphasizes a user-centric design that prioritizes data integrity and accessibility. By bridging the gap between instructional delivery and administrative oversight, the CCMS not only enhances operational efficiency but also provides a scalable and sustainable framework for modernizing the coaching experience. Ultimately, this practicum project demonstrates the practical application of full-stack web technologies in solving real-world institutional challenges, resulting in a robust, professional-grade management tool that fosters a more organized, effective, and digitally empowered educational environment.

Acknowledgments

I am deeply thankful for the invaluable blessings from the Almighty, which have made all of this possible. My sincere gratitude goes to my esteemed supervisor, Dipta Mohon Das, Lecturer in the Department of Computer Science and Engineering at the International University of Business Agriculture and Technology (IUBAT). Her guidance, encouragement, and unwavering support along with the time he invested in reviewing and evaluating this work have been truly inspiring.

I also extend my gratitude to the Department of Computer Science and Engineering (CSE) at IUBAT for their comprehensive support throughout this endeavor. In particular, I would like to thank our coordinators, Shahinur Alam and Rashedul Islam for their assistance.

Finally, my heartfelt thanks go to my parents and classmates. Their encouragement and support have been invaluable throughout this journey.

Table of Contents

Letter of Transmittal	iii
Organization's Certificate.....	iv
Student's Declaration	v
Supervisor's Certification	vi
Abstract.....	vii
Acknowledgments	viii
List of Figures.....	xiii
List of Tables	xv
Chapter 1. Introduction.....	1
1.1 Background of the Study	2
1.2 Methodology	2
1.3 Objectives	4
1.4 Process Model.....	6
1.5 Feasibility Study	7
1.5.1 Technical Feasibility.....	7
1.5.2 Economic Feasibility.	7
1.5.3 Operational Feasibility.....	8
1.6 Structure of the Report.....	8
Chapter 2. Organizational Overview	10
2.1 Organization Vision.....	11
2.2 Organization Mission.....	11
2.3 Organization Services	11
2.4 Organizational Structure	12
2.5 Address of the Organization	12
Chapter 3. Methods	13
3.1 Requirement Analysis.....	14
3.2 Requirement Engineering	14
3.2.1 User and System Requirement.....	14
3.2.2 Functional Requirements.	17

3.2.3 Non-Functional Requirements	19
3.3 Use Case Diagram.....	21
3.3.1 Use Case Diagram for CCMS.....	22
Chapter 4. Analysis	23
4.1 Activity Diagram	24
4.1.1 Activity Diagram for Teacher.....	24
4.1.2 Activity Diagram for Public User.....	25
4.1.3 Activity Diagram for Admin.....	26
4.1.4 Activity Diagram for Student.....	27
4.2 Swim-Lane Diagram.....	28
4.2.1 Swim-Lane Diagram for Public User.	28
4.2.2 Swim-Lane for Admin.	29
4.2.3 Swim-Lane for Teacher.	30
4.2.4 Swim-Lane for Student.....	31
4.3 Sequence Diagram	32
4.3.1 Sequence Diagram for Public User.....	32
4.3.2 Sequence Diagram for Admin.	33
4.3.3 Sequence Diagram for Teacher.....	34
4.3.4 Sequence Diagram for Student.	35
4.4 Class Diagram.....	35
4.4.1 Class Diagram for CCMS.....	36
Chapter 5. Project Management	37
5.1 Risk Management Process	38
5.2 Risk Identification Table.....	38
5.3 Risk Analysis Table	39
5.4 Risk Planning Table.....	40
5.5 Risk Monitoring Table.....	41
5.6 The RMMM Plan.....	42
Chapter 6. Project Planning	46
6.1 Project Planning	47
6.2 Project Estimation.....	47

6.3 Function Oriented Matrices	48
6.4 Function Point Calculation	49
6.4.1 Identifying Complexity of Transaction Function.	50
6.4.2 Identifying Complexity of Data Function.....	51
6.4.3 Unadjusted Function Point Contribution for Transaction Functions.	52
6.4.4 Unadjusted Function Point Contribution for Data Functions	54
6.4.5 Performance and Environmental Impact.	55
6.4.6 Counting Adjusted Function Point.....	56
6.5 Project Schedule Chart.....	57
6.6 Project Cost Estimation.....	58
6.6.1 Personnel Cost.	58
6.6.2 Hardware Cost.	58
6.6.3 Software Cost.....	59
6.6.4 Other Cost.	59
6.6.5 Accounts Table.	59
Chapter 7. Designing	60
7.1 Dataflow Diagram.....	61
7.1.1 Context Level DFD.....	61
7.1.2 Level 1 DFD.	62
7.1.3 Level 2 DFD.	63
7.2 Entity Relationship Diagram.....	66
7.3 Database Table Structure	68
7.4 Interface Design	71
Chapter 8. System Quality and Testing	77
8.1 System Quality Management.....	78
8.2 Software Quality Management Process	78
8.1 Software Test Cases.....	78
8.3.1 System Testing.....	79
8.3.2 System Testing Methodology.	79
8.3.3 White Box Testing.	79
8.3.4 Black Box Testing.....	80

8.3.5 Software Test Cases	80
Chapter 9. Ethical Consideration	82
8.1 Ethical Considerations in the Software Development Process	83
8.2 Sustainability in the Software Development Process	84
Chapter 10. Conclusion	86
10.1 Brief Overview of the Project	87
10.2 Proposed System Benefits	87
10.3 Limitations of the Project	87
10.4 Practicum and Its Value	87
10.5 Future Plan	88
References	89

List of Figures

Figure 1.1 Incremental Process Model	6
Figure 2.1 Organization Logo.....	11
Figure 3.1 Use Case Diagram of Coaching Center Management System.	22
Figure 4.1 Activity Diagram for Teacher.	24
Figure 4.2 Activity Diagram for Public User.....	25
Figure 4.3 Activity Diagram for Admin.	26
Figure 4.4 Activity Diagram for Student.	27
Figure 4.5 Swim-Lane Diagram for Public User	28
Figure 4.6 Swim-Lane Diagram for Admin.....	29
Figure 4.7 Swim-Lane Diagram for Teacher.....	30
Figure 4.8 Swim-Lane Diagram for Student.....	31
Figure 4.9 Sequence Diagram for Public User.	32
Figure 4.10 Sequence Diagram for Admin.	33
Figure 4.11 Sequence Diagram for Teacher.	34
Figure 4.12 Sequence Diagram for Student.....	35
Figure 4.13 Class Diagram for Coaching Center Management System.	36
Figure 6.1 Formula Table for EI and EO value	52
Figure 6.2 Formula Table for ILF and EIF value	54
Figure 6.3 Project Schedule Chart	57
Figure 7.1 Context level / Level 0 DFD	61
Figure 7.2 Level 1 DFD	62
Figure 7.3 Level 2 DFD of User Management Process	63
Figure 7.4 Level 2 DFD of Assignment Process	64
Figure 7.5 Level 2 DFD of Course Management Process	65
Figure 7.6 Level 2 DFD of Course Enrollment Process	66
Figure 7.7 ER Diagram for Coaching Center Management System.....	67
Figure 7.8 Homepage.....	71
Figure 7.9 Courses Page	71

Figure 7.10 Teacher Page	72
Figure 7.11 Contact Page.....	72
Figure 7.12 Login and Register	73
Figure 7.13 Admin Dashboard.....	73
Figure 7.14 Course CRUD Operations	74
Figure 7.15 Student List.....	74
Figure 7.16 Notice Creation.....	74
Figure 7.17 Teacher Dashboard.....	75
Figure 7.18 Teacher Course list with Batch, schedule and material upload	75
Figure 7.19 Student Dashboard.....	75
Figure 7.20 Student Course List	76
Figure 7.21 Payment System	76

List of Tables

Table 5.1 Risk Identification Table.	38
Table 5.2 Risk Analysis Table.	39
Table 5.3 Risk Planning Table.	40
Table 5.4 Risk Monitoring Table.	41
Table 5.5 RMMM for R01.	42
Table 5.6 RMMM for R03.	43
Table 5.7 RMMM for R04.	44
Table 5.8 RMMM for R06.	45
Table 6.1 Functionality, input and output table.	49
Table 6.2 Complexity Table for Transaction Functions.	50
Table 6.3 Complexity Table for Data Functions.	51
Table 6.4 Unadjusted Function Point Contribution for Transaction Functions.	52
Table 6.5 Unadjusted Function Point Contribution for Data Functions.	54
Table 6.6 Total Degree of Influence (TDI) Calculation.	55
Table 6.7 Weekly Breakdown of Project.	57
Table 6.8 Personnel cost estimation.	58
Table 6.9 Hardware cost estimation.	58
Table 6.10 Software cost estimation.	59
Table 6.11 Other cost estimation.	59
Table 6.12 Accounts Table.	59
Table 8.1 Software Test Cases.	80

Chapter 1.

Introduction

In the rapidly evolving landscape of modern education, the need for centralized digital management has become essential for institutions to maintain efficiency and transparency. The Coaching Center Management System (CCMS) is designed to meet this demand by providing a robust, high-performance platform built on the MERN stack and styled with Tailwind CSS to streamline the complexities of academic administration. By integrating distinct functionalities for Admins, Teachers, and Students, the system transforms traditional, fragmented workflows into a unified ecosystem where enrollment approvals, course scheduling, and material distribution occur seamlessly. From the public-facing landing page that showcases courses and faculty to the intricate back-end dashboards that handle payment tracking, assignment submissions, and user management, CCMS serves as a comprehensive solution for modernizing the coaching experience. Ultimately, this project is more than just a management tool; it is a scalable digital hub that enhances the productivity of educators, simplifies the journey for students, and provides administrators with the real-time oversight necessary to drive institutional growth in a digital-first world.

1.1 Background of the Study

The rapid globalization of education and the increasing competitiveness of academic environments have led to a significant surge in the demand for specialized coaching centers. However, many of these institutions still rely on manual administration, using physical registers or basic spreadsheets to track student progress, teacher schedules, and financial transactions. This traditional approach often results in data silos, communication gaps, and administrative bottlenecks that hinder the growth of both the student and the institution. During my practicum, I observed that as coaching centers scale, the lack of a centralized system makes it increasingly difficult to manage the influx of enrollments and the distribution of academic materials efficiently.

This study explores the development of the Coaching Center Management System (CCMS) as a digital solution to these operational challenges. By leveraging modern web technologies like the MERN stack, the project aims to bridge the gap between administrative oversight and classroom delivery. The current state of educational management demands platforms that are not only functional but also accessible and real-time; thus, this practicum focuses on

creating a multi-user ecosystem where students can track their learning journey, teachers can manage their curricula, and admins can maintain institutional integrity through a single, cohesive interface. Ultimately, this work is grounded in the belief that digital transformation in coaching centers is essential for creating a more organized, transparent, and effective learning environment for the next generation of students.

1.2 Methodology

The development of the Coaching Center Management System and the compilation of this report were guided by a systematic approach to data collection and technical implementation. To ensure the platform met real-world requirements, I gathered insights from two distinct types of data sources, which allowed me to balance practical user needs with established industry standards for web development.

The primary data for this project was gathered through direct engagement and hands-on experimentation. This firsthand information was crucial in shaping the system's architecture and ensuring the user interface remained intuitive for students and teachers alike. The key primary sources included:

- User Requirement Gathering: Conducting informal discussions with potential users including students and educators to identify the most common pain points in existing coaching center workflows.
- Direct Observation: Analyzing the manual administrative processes of local coaching centers to understand how enrollment requests and payment tracking are currently handled.
- Iterative Testing & Debugging: Frequent internal testing of the MERN stack components to verify that the Admin, Teacher, and Student dashboards functioned correctly under various scenarios.

- **System Feedback:** Reviewing the output of real-time data handling, such as payment status updates and notice publishing, to ensure data integrity.

Secondary data sources provided the theoretical framework and technical guidance necessary to build a secure and scalable application. These sources helped in troubleshooting complex code and following best practices for full-stack development. The secondary sources utilized include:

- **Technical Documentation:** Deep diving into the official documentation for MongoDB, Express.js, React, and Node.js to implement optimized backend logic and frontend state management.
- **Tailwind CSS Framework Guides:** Referencing styling documentation to ensure a responsive and modern design consistent with contemporary web standards.
- **Academic Journals and Articles:** Reviewing existing literature on Management Information Systems (MIS) and educational technology to understand the broader impact of digitalization on learning environments.
- **Online Developer Communities:** Utilizing platforms like Stack Overflow, GitHub, and medium-based tech blogs to solve specific architectural challenges and stay updated on the latest MERN stack security patches.

1.3 Objectives

The objectives of this project are centered on bridging the gap between traditional educational management and modern technological solutions. By defining both a high-level vision and granular functional goals, the project ensures that the resulting system is both purposeful and technically sound.

The primary objective of this practicum is to design and develop a comprehensive, full-stack Coaching Center Management System (CCMS) that digitizes and centralizes the

administrative, instructional, and student-related operations of a coaching institution. The project aims to replace manual, error-prone processes with an automated MERN-based ecosystem that enhances operational efficiency, ensures data transparency, and provides seamless user experience for administrators, teachers, and students alike.

To achieve the broad vision of the project, the following specific objectives have been established:

- **Develop a Role-Based Access Control (RBAC) System:** To implement a secure authentication and authorization layer that provides distinct, customized dashboards for Admins, Teachers, and Students, ensuring data privacy and relevant functionality for each user type.
- **Streamline Enrollment and Financial Tracking:** To create an integrated workflow where administrators can review student enrollment requests and verify payment statuses in real-time, reducing the friction associated with manual registration.
- **Facilitate Academic Resource Management:** To empower teachers with tools to manage their specific courses, including the ability to create batches, schedule classes, upload study materials, and assign coursework through a digital interface.
- **Enhance Communication and Transparency:** To build a centralized notice board and contact system that allows administrators to broadcast official updates to all users and receive direct inquiries from the public.
- **Implement Responsive and Modern UI:** To utilize Tailwind CSS to design an intuitive, mobile-friendly interface that requires minimal technical training for users while maintaining a professional and engaging aesthetic.

1.4 Process Model

For the development of the Coaching Center Management System, I utilized the Incremental Process Model. This approach was chosen to handle the project's complexity by breaking it down into smaller, manageable segments or "increments." Instead of delivering the entire system at once, I developed the core functionalities first such as the landing page and user authentication and then progressively added more sophisticated features, like the Admin dashboard, Teacher tools, and Student enrollment tracking. This allowed me to maintain a clear roadmap over the 12-week duration, ensuring that each part of the MERN stack was stable before building the next set of features on top of it.

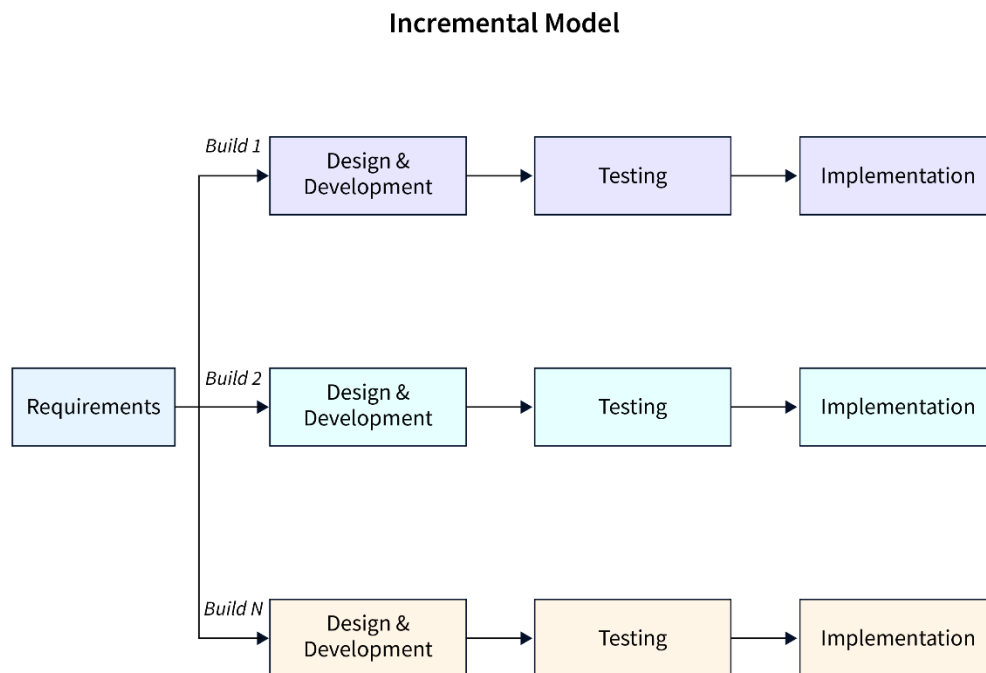


Figure 1.1 Incremental Process Model

The reasons for choosing the Incremental Process Model for this project are:

- **Systematic Feature Delivery:** It allowed me to prioritize the most critical parts of the system first, ensuring that the foundational user login and registration worked perfectly before moving on to complex data management.

- **Reduced Risk of Failure:** By completing and testing the project in increments, I could identify technical hurdles early in the 12-week timeline rather than facing a mountain of bugs during the final week.
- **Ease of Management for a Solo Developer:** As I was working alone, focusing on one "increment" at a time (e.g., finishing the Teacher Dashboard entirely before starting the Student Dashboard) kept the workload from becoming overwhelming.
- **Tangible Progress:** This model provided a sense of accomplishment throughout the practicum, as I had a functional version of the software at the end of every stage that I could review and refine.
- **Flexibility in Refinement:** If a specific feature, like the payment status logic, needed adjustment, I could refine that specific increment without having to redesign the modules I had already completed.

1.5 Feasibility Study

Before proceeding with the development, I conducted a feasibility study to ensure the project was practical, sustainable and capable of meeting real-world demands. The system proved to be viable across the following three key dimensions:

1.5.1 Technical Feasibility

The project is technically viable as it leverages the MERN stack, a mature and widely supported technology suite. Using JavaScript across the entire stack ensures a seamless flow between the database and the UI. With the help of Tailwind CSS, I can build a responsive, high-performance interface that works on any device. The availability of robust documentation and community support for these tools ensures that all complex features, such as secure authentication and real-time data updates, are well within technical reach.

1.5.2 Economic Feasibility

From a cost perspective, the system is highly efficient as it is built using open-source technologies, eliminating expensive licensing fees. Since the development was handled individually during the practicum, the primary investment was time rather than capital. For a coaching center, the system offers long-term savings by automating manual administrative tasks like payment tracking and notice distribution thereby reducing the need for physical resources and dedicated administrative staff.

1.5.3 Operational Feasibility

Operationally, the system is designed to integrate effortlessly into the daily routine of an educational center. The interface is intuitive, meaning admins, teachers, and students can navigate their dashboards with minimal training. By solving specific pain points—such as transparent enrollment statuses and centralized material sharing the platform provides clear value to its users, ensuring high adoption rates and a smooth transition from traditional methods to a digital workflow.

1.6 Structure of the Report

This report is structured into ten comprehensive chapters that document the end-to-end journey of developing the Coaching Center Management System, from initial concept to final ethical evaluation. The following outline provides a roadmap of the chapters included in this practicum:

- Chapter 1: Introduction – Sets the stage by providing the background, objectives, and the incremental methodology used to build the system, along with its feasibility.
- Chapter 2: Organization Overview – Introduces the institutional context, covering the vision, mission, and professional structure of the organization where the practicum was conducted.
- Chapter 3: Requirement Engineering – Breaks down the specific needs of the system, including functional and non-functional requirements, and visualizes user interactions through Use Case diagrams.

- Chapter 4: Analysis – Delves into the technical logic of the application using Activity, Swim-lane, and Class diagrams to map out how data and users move through the system.
- Chapter 5 & 6: Project Management and Planning – Details the "behind-the-scenes" work, including risk mitigation (RMMM plan), complexity estimation using Function Point analysis, and the financial budgeting for hardware and software.
- Chapter 7: Designing – Focuses on the blueprint of the system, showcasing the user interface design, Data Flow Diagrams (DFD), and the structural normalization of the database.
- Chapter 8: System Quality and Testing – Documents the quality assurance process, outlining the testing standards and specific test cases used to ensure the MERN stack application is bug-free and reliable.
- Chapter 9: Ethical Consideration – Reflects on the broader impact of the software, discussing data privacy, intellectual property, and the social and economic sustainability of the project.
- Chapter 10: Conclusion – Summarizes the project's achievements, acknowledges its current limitations, and explores the personal value gained from the practicum experience alongside future plans for the system.

Chapter 2.

Organizational Overview

2.1 Organization Vision

To be the most trusted and innovative educational partner in Bangladesh, empowering students with the linguistic and academic skills necessary to excel in a globalized world. Career Point Saifur's envisions a society where every learner has access to high-quality instructions that bridge the gap between their potential and their career goals.



Figure 2.1 Organization Logo

2.2 Organization Mission

- Empowerment through Language: To provide world-class English language training that removes communication barriers for students and professionals.
- Academic Excellence: To deliver comprehensive coaching for competitive exams (BCS, Bank Jobs, IELTS) using proven, student-centric methodologies.
- Innovation: To integrate modern technology into the learning experience such as digitized management systems to enhance administrative efficiency and student satisfaction.

2.3 Organization Services

Career Point Saifur's offers a wide array of educational services tailored to different age groups and career paths:

- Language Programs: Spoken English, Phonetics, and Freehand Writing.
- Test Preparation: IELTS, TOEFL and SAT coaching.

- Competitive Exams: Comprehensive preparation for BCS (Preliminary & Written) and Bank Job recruitment.
- Junior Programs: Specialized English courses designed for kids and school-level students.
- Corporate Training: Tailored communication workshops for working professionals.

2.4 Organizational Structure

The branch operates under a hierarchical structure to ensure academic quality and administrative smooth-flow:

1. Branch Manager: Oversees the entire operation, strategic planning, and financial management.
2. Academic Coordinator: Manages the teaching staff, curriculum delivery, and class scheduling.
3. Instructors/Teachers: Subject matter experts responsible for classroom delivery and student assessment.
4. Administrative & Support Staff: Handles student admissions, front-desk inquiries, payments, and notice maintenance.
5. Technical Support (Intern/Developer): Manages the digital infrastructure and system development (this is where your project fits).

2.5 Address of the Organization

Career Point Saifur's

Rahman Shopping Mall (2nd Floor),

Chandana Chowrasta,

Gazipur - 1702, Bangladesh.

Chapter 3.

Requirement Engineering

3.1 Requirement Analysis

Software engineers design modules for the data, functional, and behavioral domains during the requirement analysis stage of software development. Designers may better see the data, functionality and behavior of the system with the aid of this method. By considering the needs of stakeholders and making sure they are quantifiable, testable and in line with business requirements it creates standards.

3.2 Requirement Engineering

The main goals of requirements engineering are to define software systems and determine user needs. The first step is to comprehend user expectations and how system design is impacted by their needs. This area makes sure the created systems satisfy the needs of the users.

3.2.1 User and System Requirement

UR 1: (Public User – Information Access and Communication)

The system shall allow public users to explore information about the coaching center and communicate with the admin without requiring registration or login.

SR 1: Specifications for System Requirement for Public User

- 1.1: The system shall provide unrestricted access to the home/landing page for all visitors.
- 1.2: The system shall display a list of available courses on the Courses page.
- 1.3: The system shall allow users to view a list of teachers on the Teachers page.
- 1.4: The system shall display notices published by the admin on the Notices page.
- 1.5: The system shall provide an About page containing general information about the

coaching center.

1.6: The system shall provide a Contact page where users can submit messages to the admin.

1.7: The system shall store submitted messages and make them accessible in the admin dashboard.

UR 2: (Admin – System Management and Control)

The system shall provide the admin with full control over users, courses, enrollments, and system content to ensure smooth operation of the coaching center.

SR 2: Specifications for System Requirement for Admin

2.1: The system shall provide the admin with a secure login and access to a dedicated dashboard.

2.2: The dashboard shall display total numbers of students, teachers, and courses.

2.3: The system shall display recent system activities or updates on the dashboard.

2.4: The system shall allow the admin to create new courses and assign teachers to them.

2.5: The system shall allow the admin to edit and delete existing courses.

2.6: The system shall allow the admin to view all enrollment requests submitted by students.

2.7: The system shall display payment status (Paid/Not Paid) for each enrollment request.

2.8: The system shall allow the admin to approve or reject student enrollment requests.

2.9: The system shall allow the admin to view, block, or delete student accounts.

2.10: The system shall allow the admin to create, view, block, or delete teacher accounts.

2.11: The system shall allow the admin to publish notices visible to all users.

2.12: The system shall allow the admin to view messages submitted through the contact page.

2.13: All administrative functionalities shall be protected through secure authentication and authorization.

UR 3: (Teacher – Course Delivery and Student Interaction)

The system shall provide teachers with tools to manage assigned courses, organize class activities, and interact with enrolled students.

SR 3: Specifications for System Requirement for Teacher

3.1: The system shall allow teachers to log in securely using credentials provided by the admin.

3.2: The system shall display a dashboard showing total assigned courses, students, batches, and materials.

3.3: The system shall allow teachers to view all assigned courses in the “My Courses” section.

3.4: The system shall allow teachers to create and manage batches for each course.

3.5: The system shall allow teachers to create and manage class schedules.

3.6: The system shall allow teachers to upload course materials such as documents and resources.

3.7: The system shall allow teachers to view the list of students enrolled in each course.

3.8: The system shall allow teachers to create assignments for students within specific courses.

3.9: Access to teacher functionalities shall be restricted through secure authentication.

UR 4: (Student – Enrollment, Learning, and Assignment Submission)

The system shall allow students to register, enroll in courses, access learning materials, and submit assignments while monitoring their enrollment status.

SR 4: Specifications for System Requirement for Student

- 4.1: The system shall allow new users to register as students by providing required information.
- 4.2: The system shall allow registered students to log in securely to their dashboard.
- 4.3: The system shall display a dashboard summarizing applied, pending, accepted, and rejected courses.
- 4.4: The system shall allow students to browse and apply for available courses.
- 4.5: The system shall provide a payment option during course enrollment and record the payment status.
- 4.6: The system shall allow students to view all enrolled courses in the “My Courses” section.
- 4.7: The system shall allow students to view course details, including schedule, batch number, and materials.
- 4.8: The system shall maintain a list of all enrollment requests along with their status (Approved/Pending).
- 4.9: The system shall allow students to upload assignments for their respective courses.
- 4.10: Access to all student functionalities shall require secure authentication.

3.2.2 Functional Requirements

Functional requirements describe what the system does, the actual features and operations.

1. User Management

FR1.1: The system shall allow students to register and create accounts.

FR1.2: The system shall allow users (Admin, Teacher, Student) to log in securely.

FR1.3: The system shall allow the admin to create teacher accounts.

FR1.4: The system shall allow the admin to block or delete student and teacher accounts.

2. Course Management

FR2.1: The system shall allow the admin to create courses and assign teachers.

FR2.2: The system shall allow the admin to edit and delete courses.

FR2.3: The system shall display available courses to public users and students.

3. Enrollment Management

FR3.1: The system shall allow students to apply for courses.

FR3.2: The system shall store enrollment requests with payment status.

FR3.3: The system shall allow the admin to approve or reject enrollment requests.

FR3.4: The system shall display enrollment status (pending/approved/rejected) to students.

4. Payment Handling

FR4.1: The system shall provide a payment option during course enrollment.

FR4.2: The system shall record payment status as “Paid” or “Not Paid.”

FR4.3: The system shall allow the admin to view payment status for each student.

5. Teacher Functionalities

FR5.1: The system shall allow teachers to view assigned courses.

FR5.2: The system shall allow teachers to create batches for courses.

FR5.3: The system shall allow teachers to create and manage class schedules.

FR5.4: The system shall allow teachers to upload course materials.

FR5.5: The system shall allow teachers to view students enrolled in their courses.

FR5.6: The system shall allow teachers to create assignments.

6. Student Functionalities

FR6.1: The system shall display a personalized dashboard for each student.

FR6.2: The system shall allow students to view enrolled courses.

FR6.3: The system shall allow students to access schedules, batches, and materials.

FR6.4: The system shall allow students to upload assignments.

7. Notice and Communication

FR7.1: The system shall allow the admin to publish notices.

FR7.2: The system shall display notices to all users and public visitors.

FR7.3: The system shall allow public users to send messages via the contact form.

FR7.4: The system shall store and display messages in the admin dashboard.

8. Dashboard and Reporting

FR8.1: The system shall display summary statistics (students, teachers, courses) on the admin dashboard.

FR8.2: The system shall display course and enrollment summaries on student and teacher dashboards.

FR8.3: The system shall show recent activities or updates in the admin dashboard.

3.2.3 Non-Functional Requirements

Non-functional requirements describe how well the system performs.

1. Performance

NFR1.1: The system should load pages within 2–3 seconds under normal usage.

NFR1.2: The system should support multiple users accessing it simultaneously without major delays.

2. Security

NFR2.1: The system shall require secure authentications for all registered users.

NFR2.2: User passwords shall be stored in encrypted/hashed format.

NFR2.3: The system shall restrict access based on user roles (Admin, Teacher, Student).

NFR2.4: The system shall protect user data from unauthorized access.

3. Usability

NFR3.1: The system shall provide a simple and user-friendly interface.

NFR3.2: Navigation between pages shall be clear and consistent.

NFR3.3: The system shall be usable by users with basic computer knowledge.

4. Reliability

NFR4.1: The system should operate without failure during normal use.

NFR4.2: The system shall ensure data consistency during operations like enrollment and payment updates.

5. Availability

NFR5.1: The system should be available 24/7 for users.

NFR5.2: Downtime should be minimized and properly communicated if maintenance occurs.

6. Scalability

NFR6.1: The system should be able to handle increasing numbers of users, courses, and data over time.

7. Maintainability

NFR7.1: The system should be designed in a modular way to allow easy updates and maintenance.

NFR7.2: Errors and bugs should be easy to identify and fix.

8. Compatibility

NFR8.1: The system should be accessible via modern web browsers (Chrome, Edge, Firefox).

NFR8.2: The system should be responsive and usable on mobile, tablet, and desktop devices.

3.3 Use Case Diagram

A Use Case Diagram is a type of Unified Modeling Language (UML) diagram that represents the interaction between actors (users or external systems) and a system under consideration to accomplish specific goals. It provides a high-level view of the system's functionality by illustrating the various ways users can interact with it.

3.3.1 Use Case Diagram for Coaching Center Management System(CCMS)

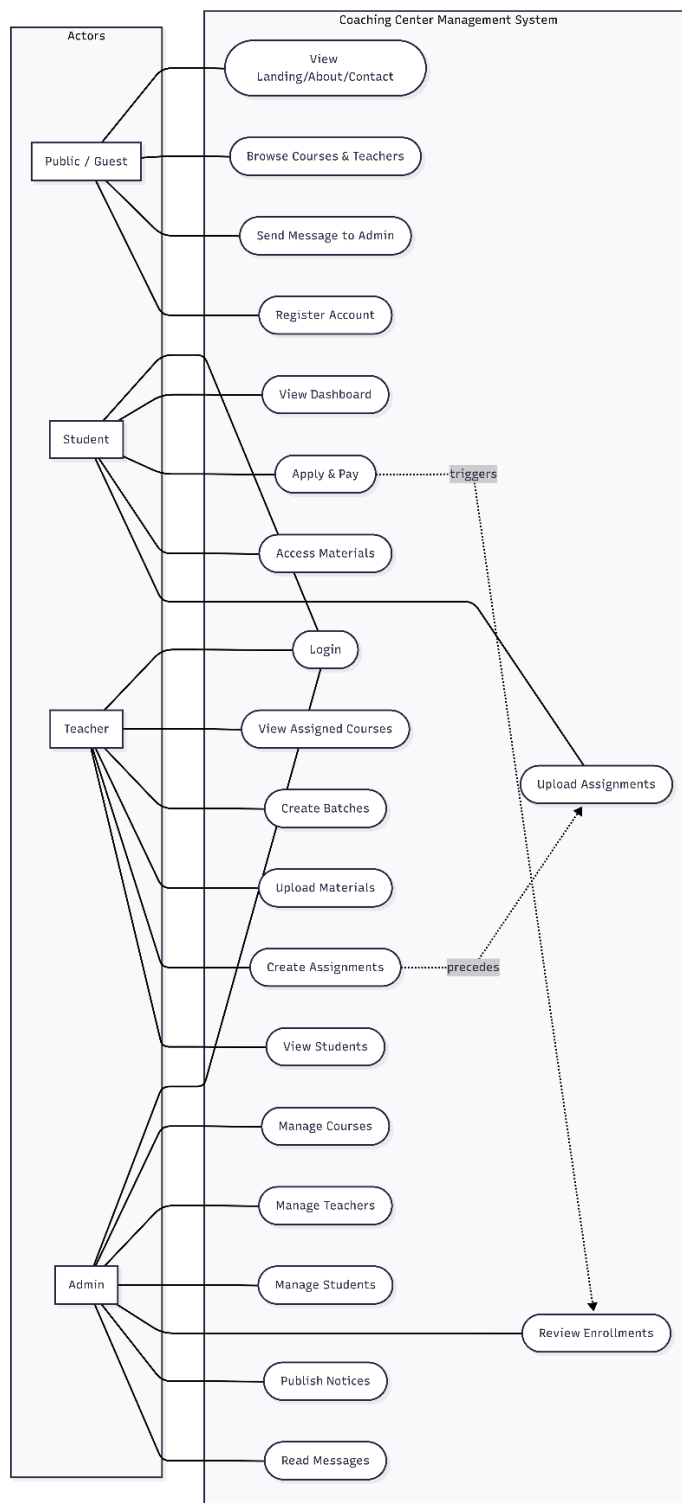


Figure 3.1: Use Case Diagram of Coaching Center Management System

Chapter 4.

Analysis

4.1 Activity Diagram

An activity diagram is a type of behavioral diagram used in UML (Unified Modeling Language) to represent the flow of activities or actions in a system. It illustrates how tasks, decisions and processes are coordinated to achieve a specific goal. The diagram includes activities (represented as ovals), decision points (diamonds) and the flow of control (arrows). It's commonly used to model workflows, business processes or use case scenarios helping visualize sequential or parallel activities, their conditions and transitions between them.

4.1.1 Activity Diagram for Teacher

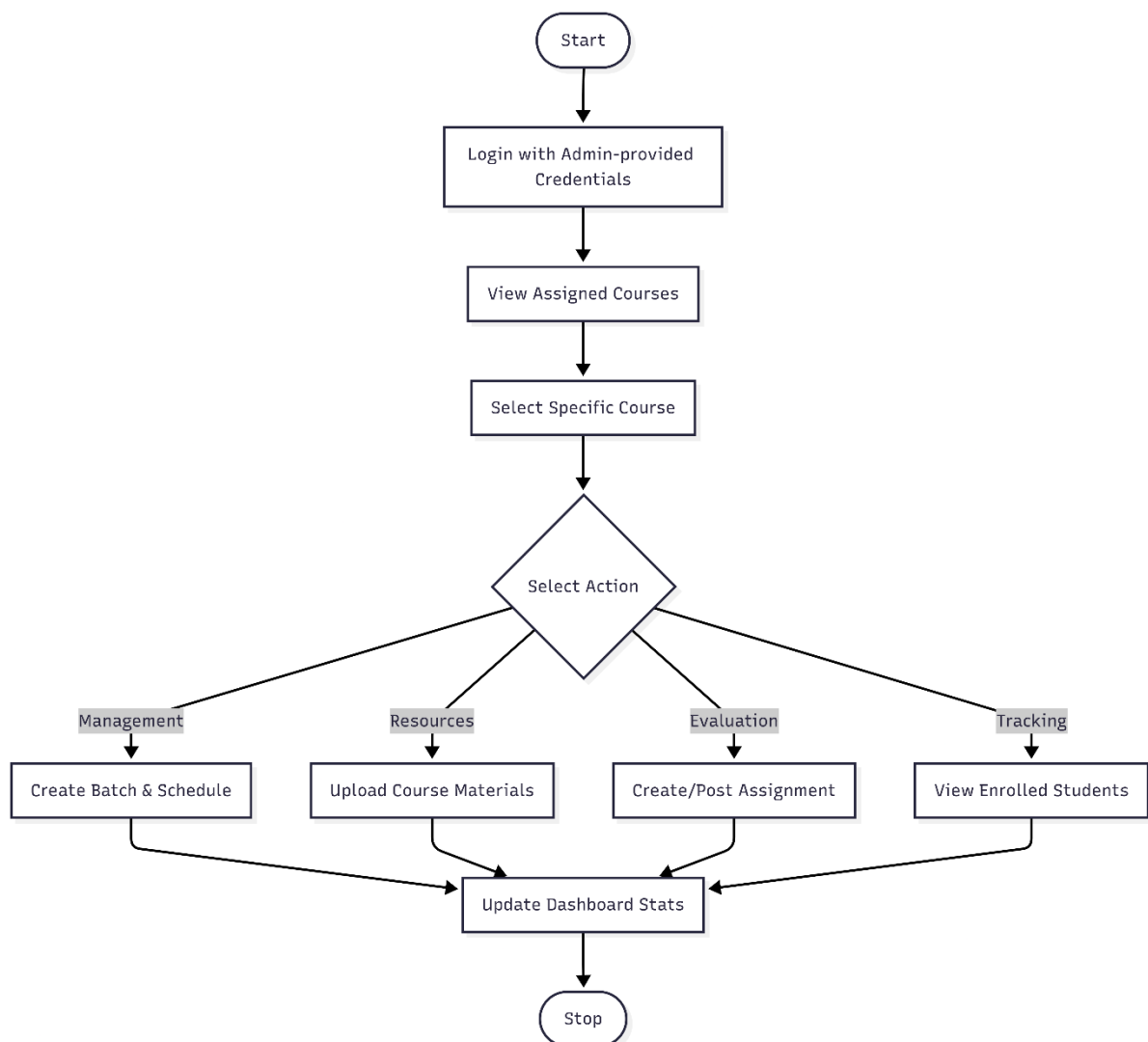


Figure 4.1 Activity Diagram for Teacher

4.1.2 Activity Diagram for Public User

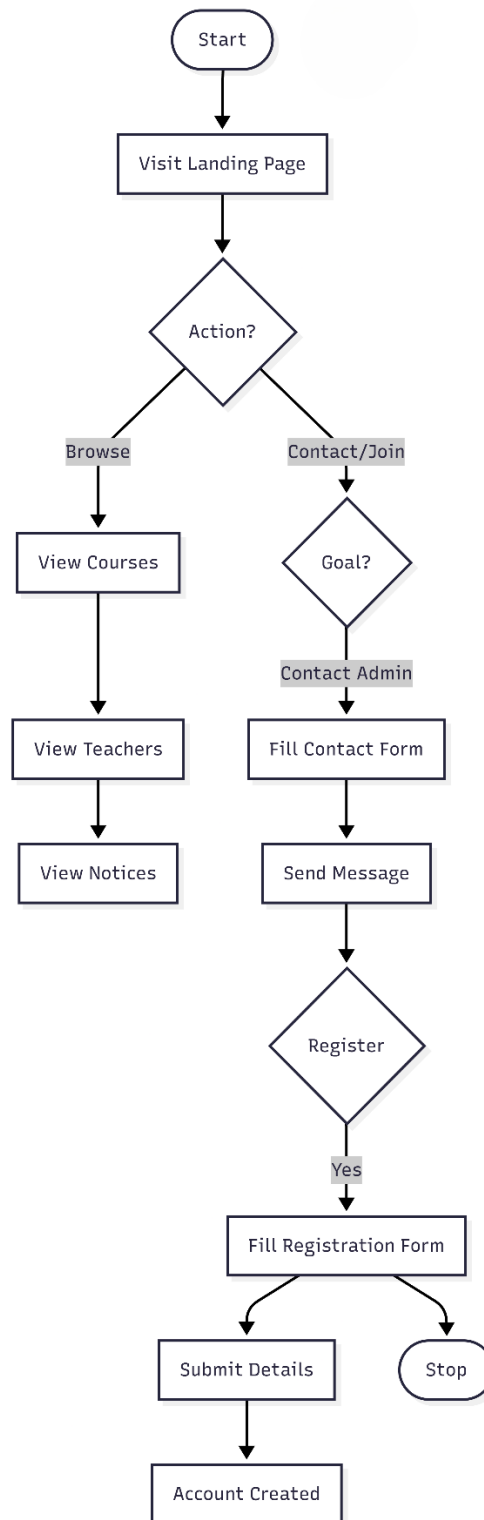


Figure 4.2 Activity Diagram for Public User

4.1.3 Activity Diagram for Admin

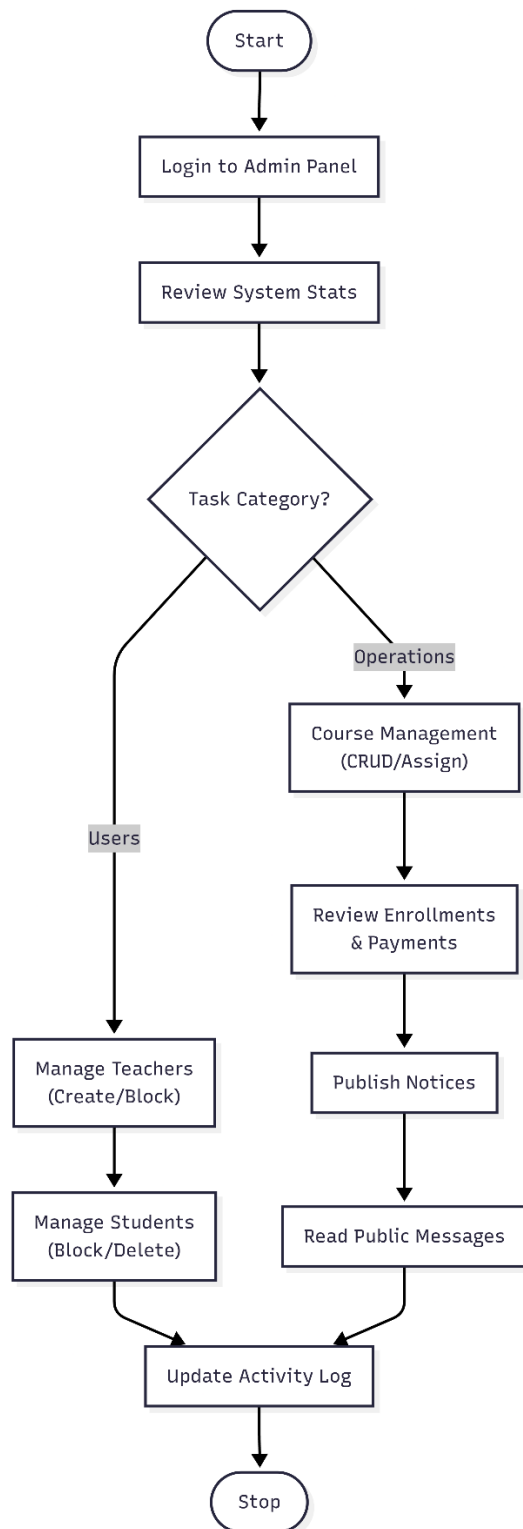


Figure 4.3 Activity Diagram for Admin

4.1.4 Activity Diagram for Student

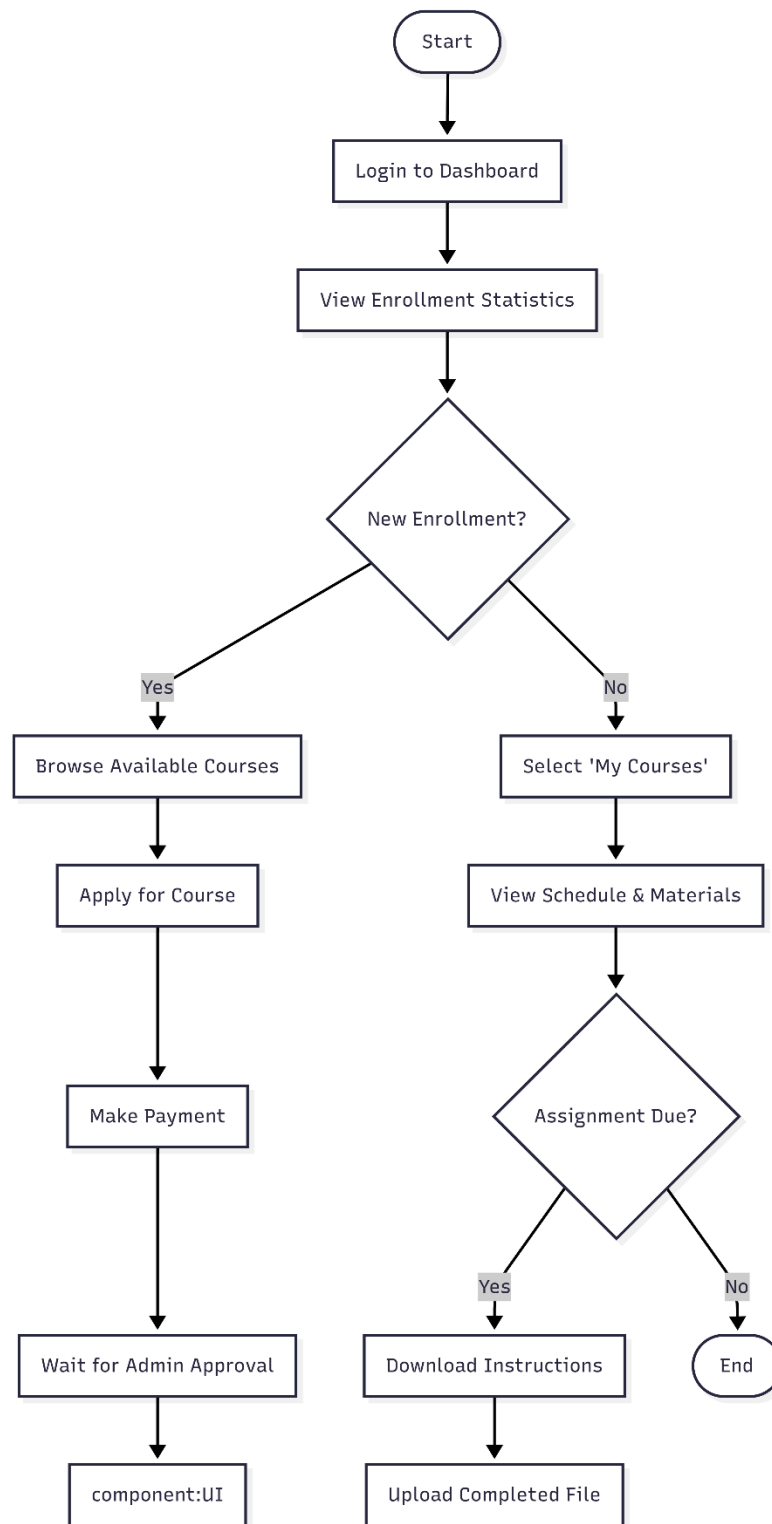


Figure 4.4 Activity Diagram for Student

4.2 Swim-Lane Diagram

A swim lane diagram is a type of flowchart that visually distinguishes tasks, processes or activities across different departments, roles or stakeholders within an organization. Each lane represents a particular group or entity responsible for specific parts of the process, helping to clarify responsibilities and improve workflow understanding.

4.2.1 Swim-Lane Diagram for Public User

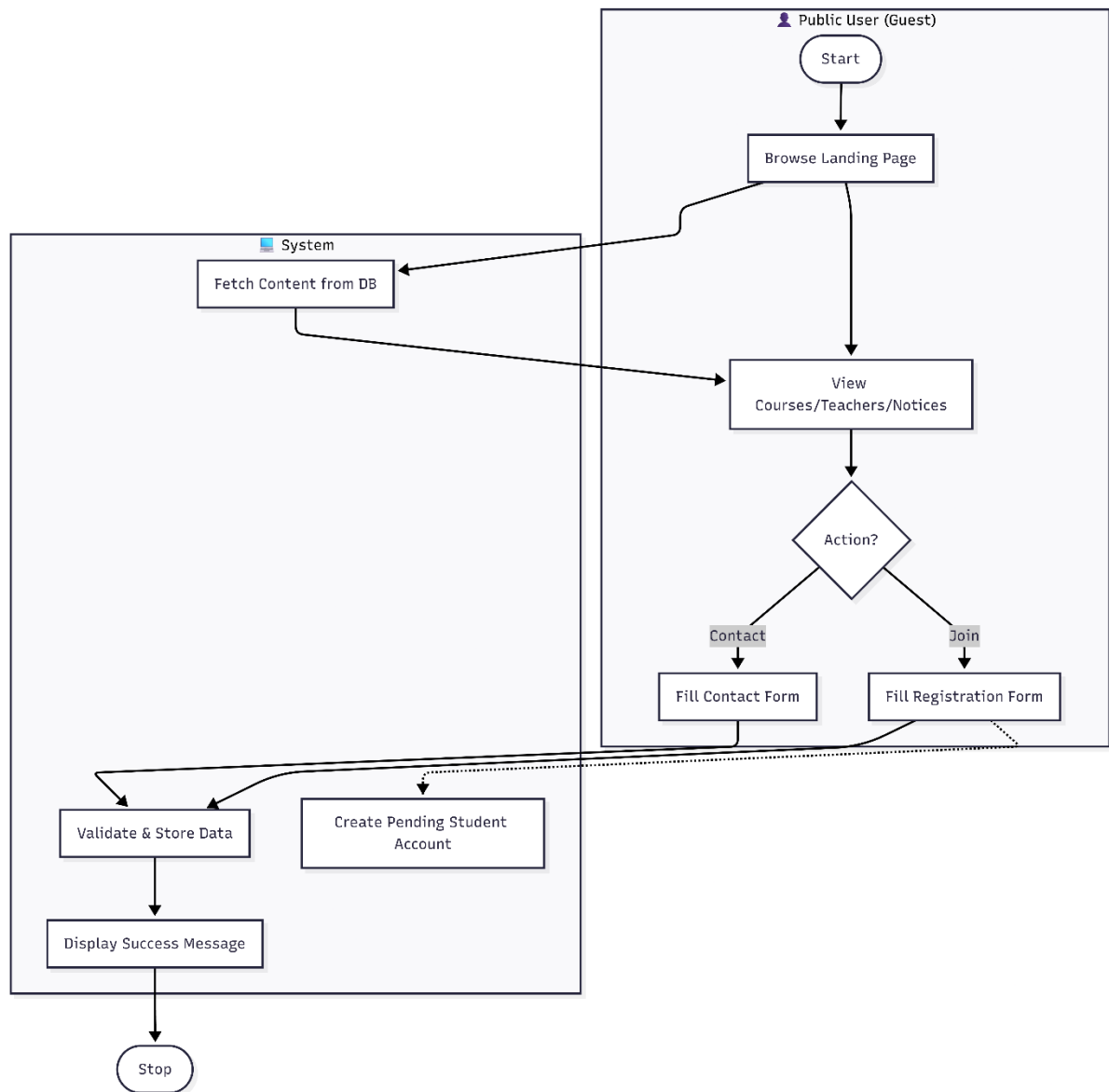


Figure 4.5 Swim-Lane Diagram for Public User

4.2.2 Swim-Lane Diagram for Admin

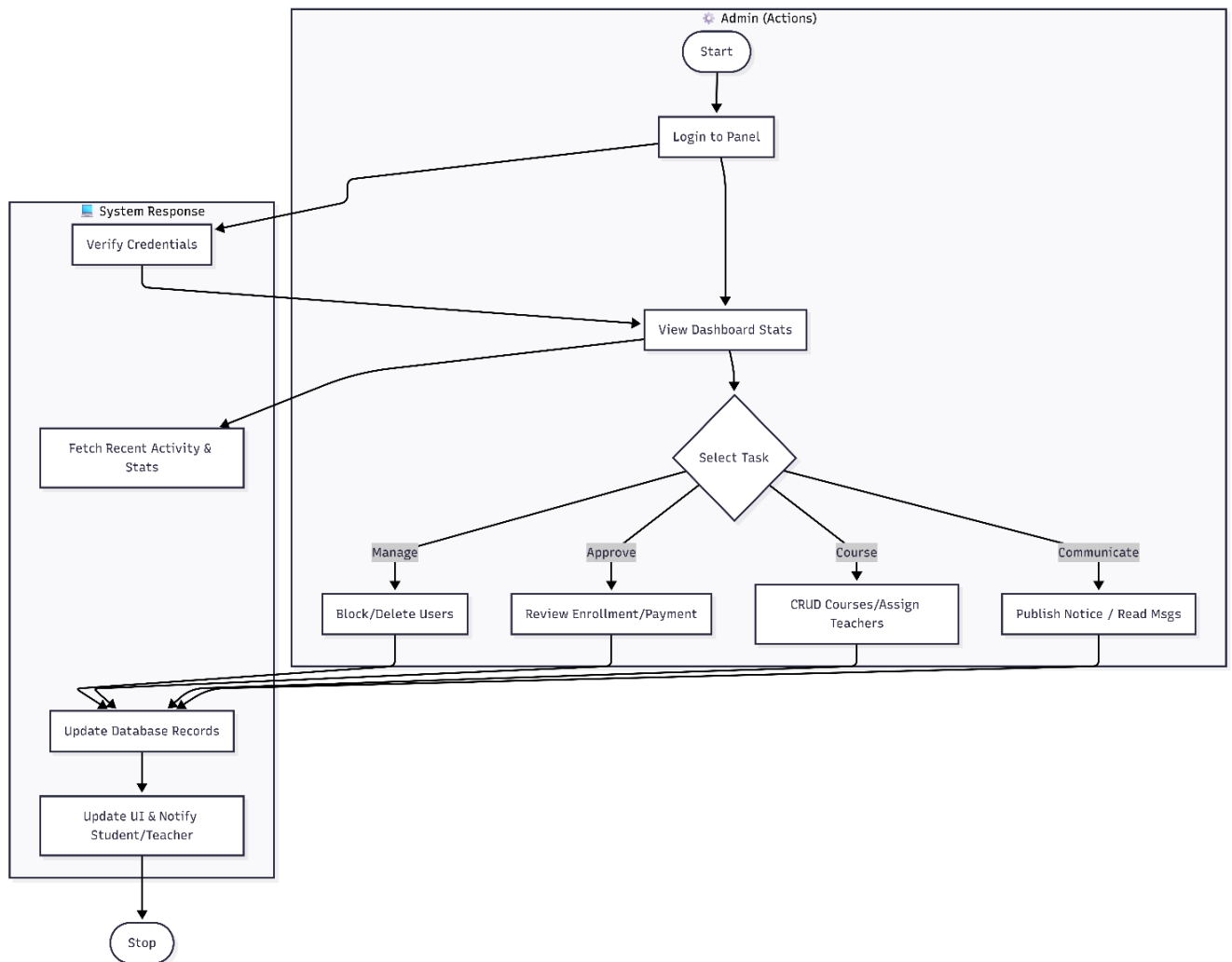


Figure 4.6 Swim-Lane Diagram for Admin

4.2.3 Swim-Lane Diagram for Teacher

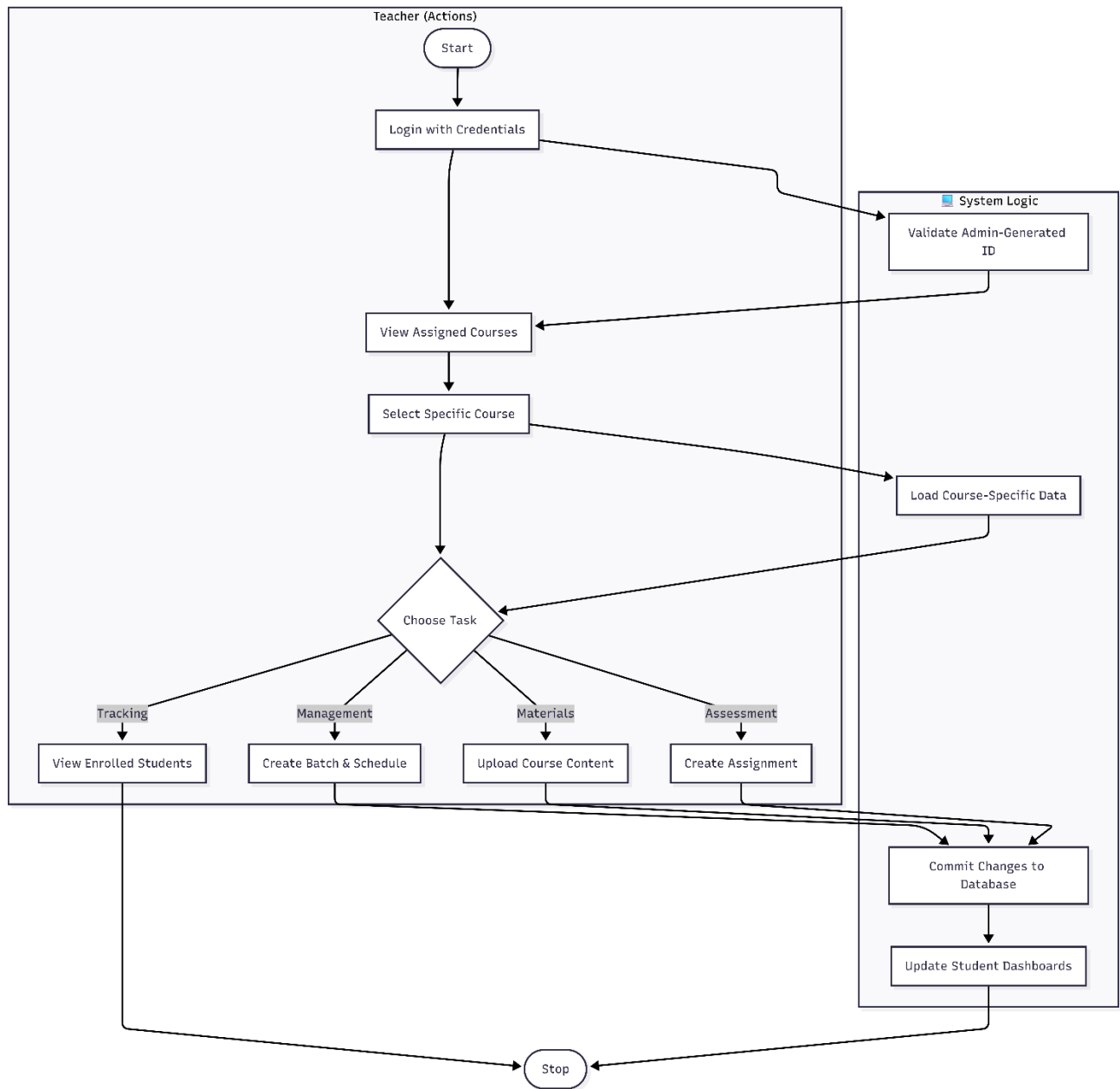


Figure 4.7 Swim-Lane Diagram for Teacher

4.2.4 Swim-Lane Diagram for Student

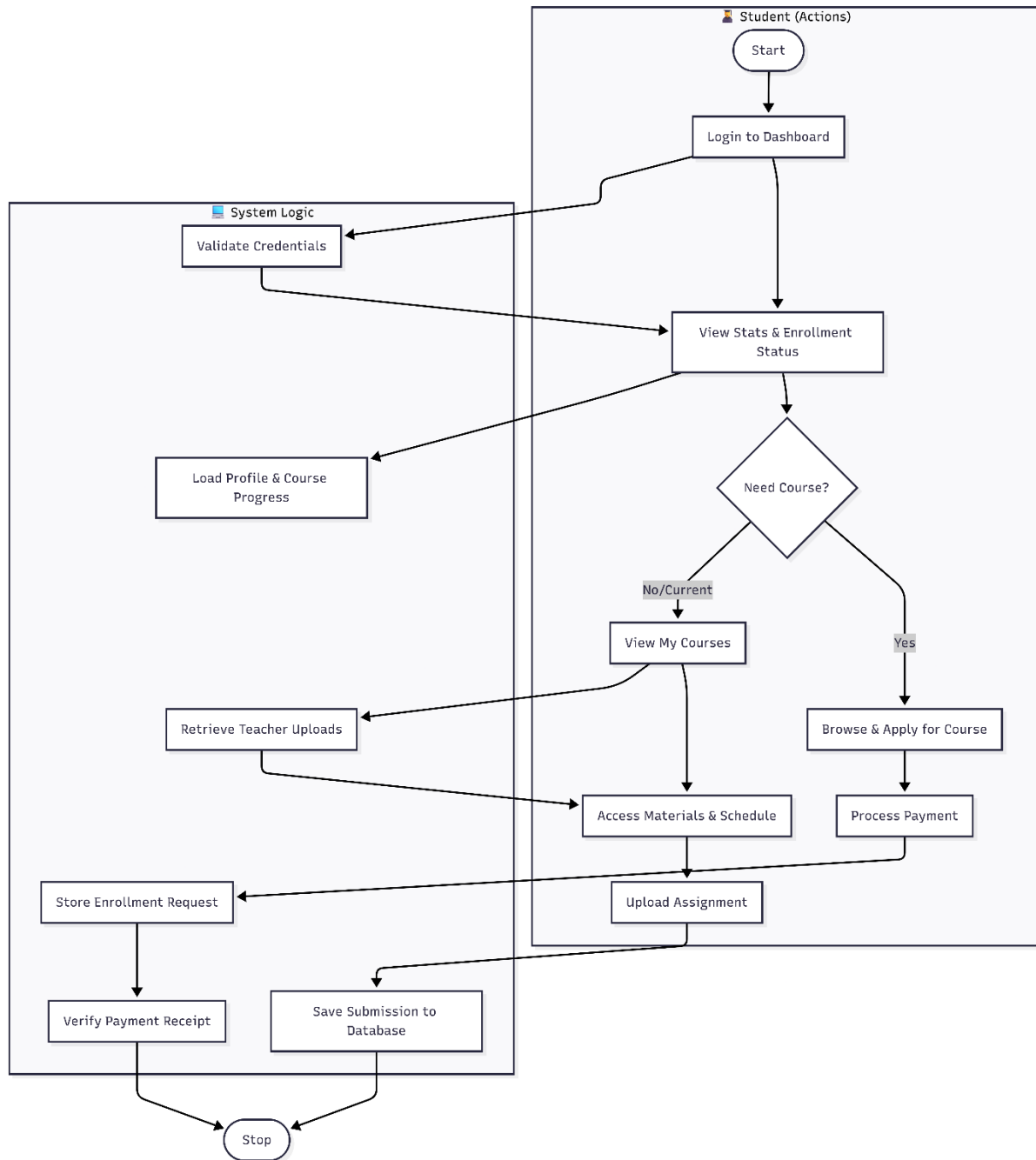


Figure 4.8 Swim-Lane Diagram for Student

4.3 Sequence Diagram

A sequence diagram is a type of interaction diagram used in system design to visually represent how objects or components interact with each other over time in a particular scenario. It shows the flow of messages or events between various actors (users or systems) and objects in sequential order.

4.3.1 Sequence Diagram for Public User

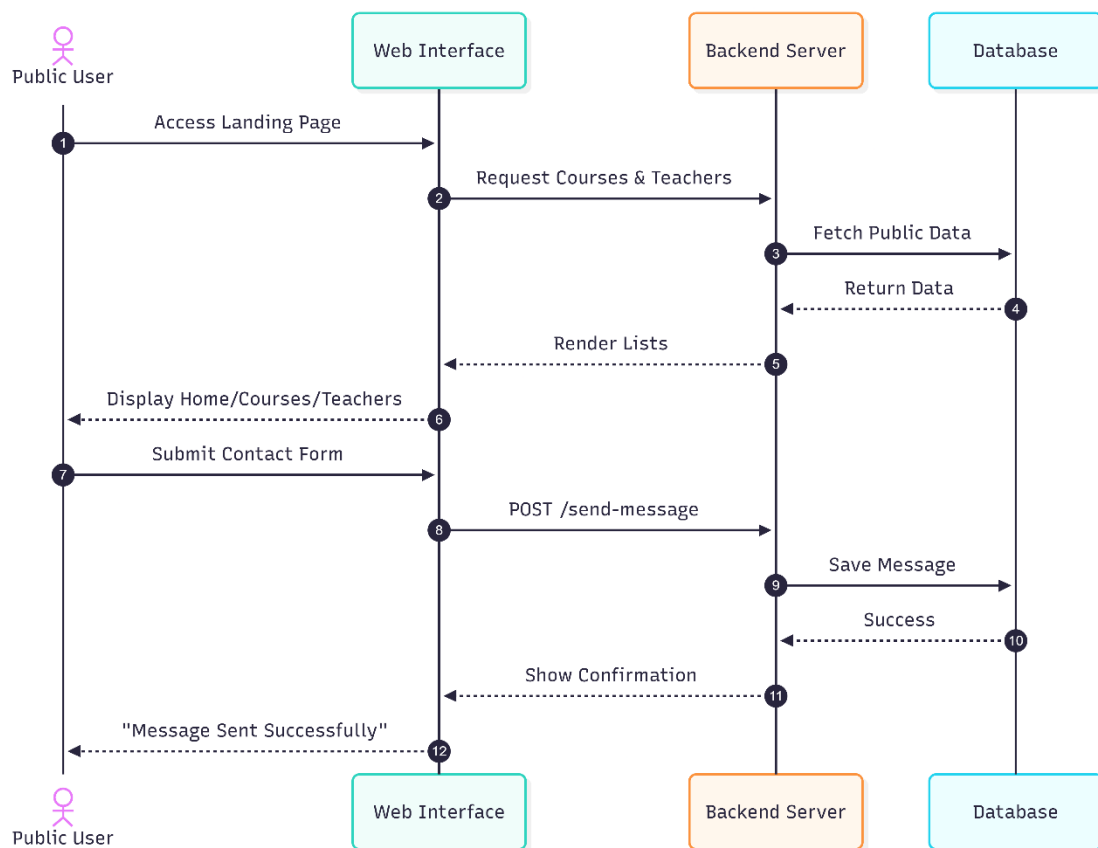


Figure 4.9 Sequence Diagram for Public User

4.3.2 Sequence Diagram for Admin

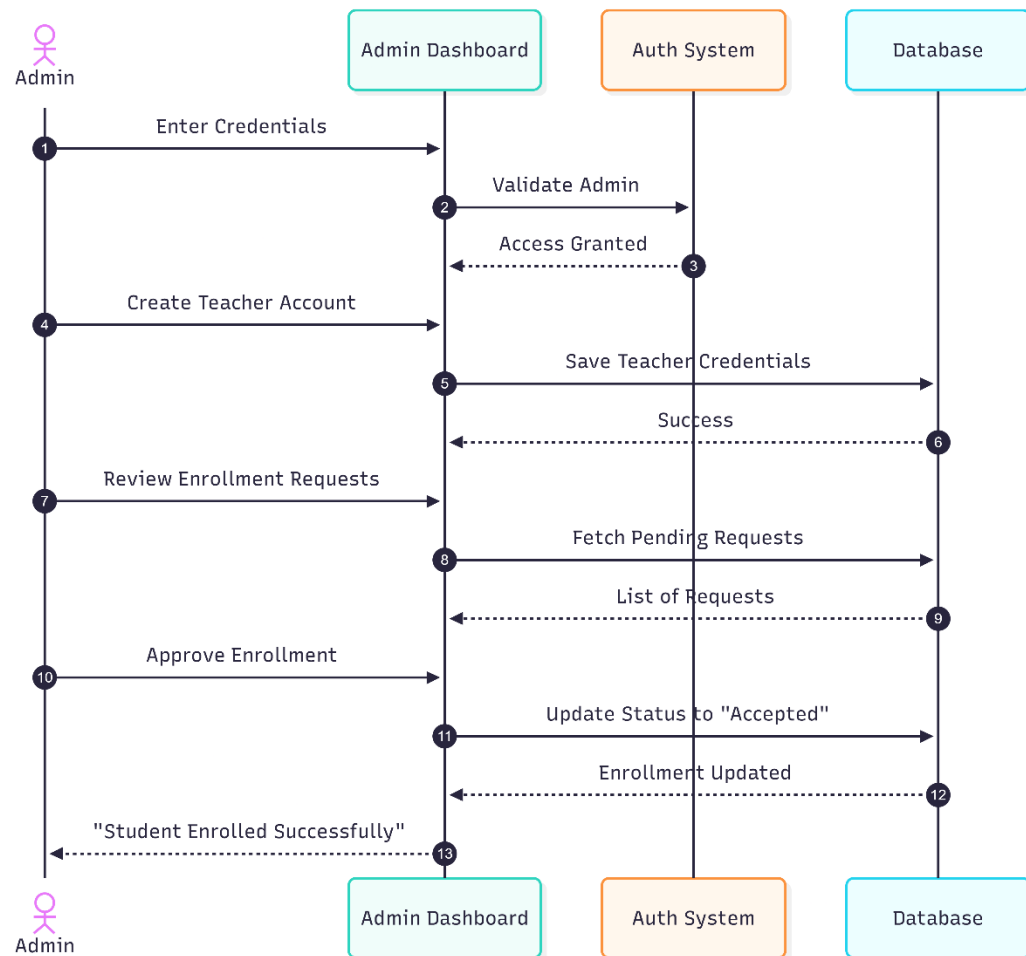


Figure 4.10 Sequence Diagram for Admin

4.3.3 Sequence Diagram for Teacher

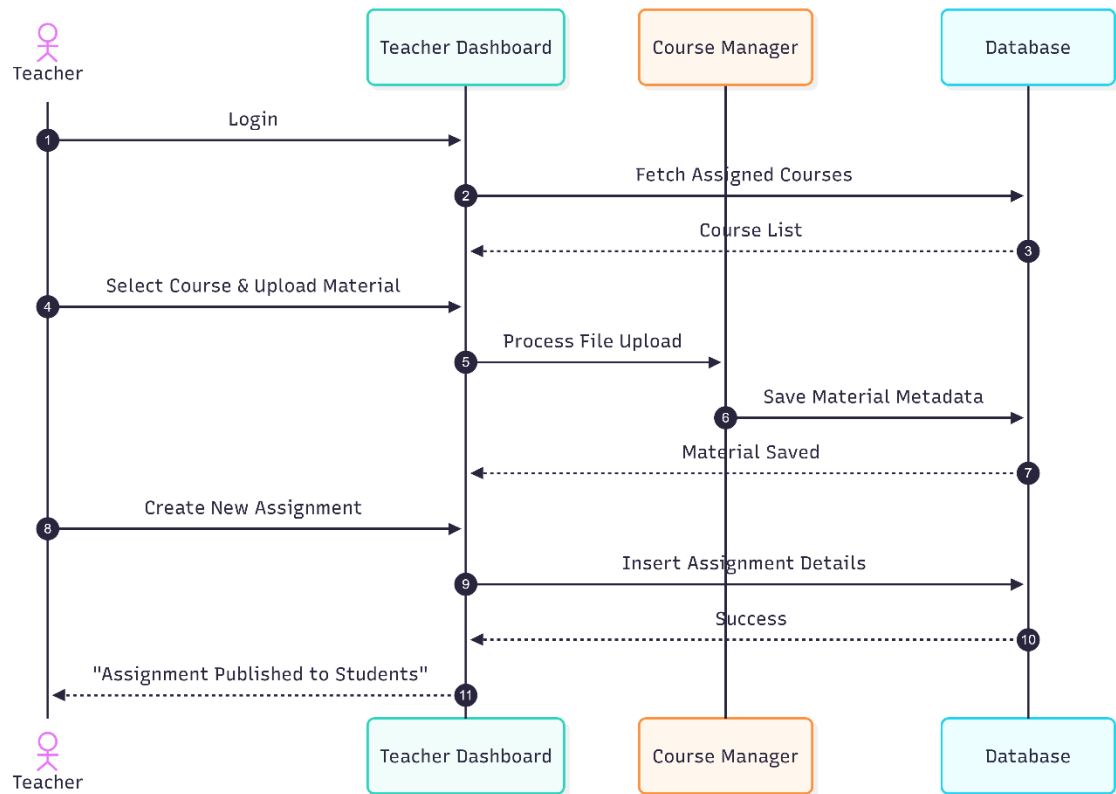


Figure 4.11 Sequence Diagram for Teacher

4.3.4 Sequence Diagram for Student

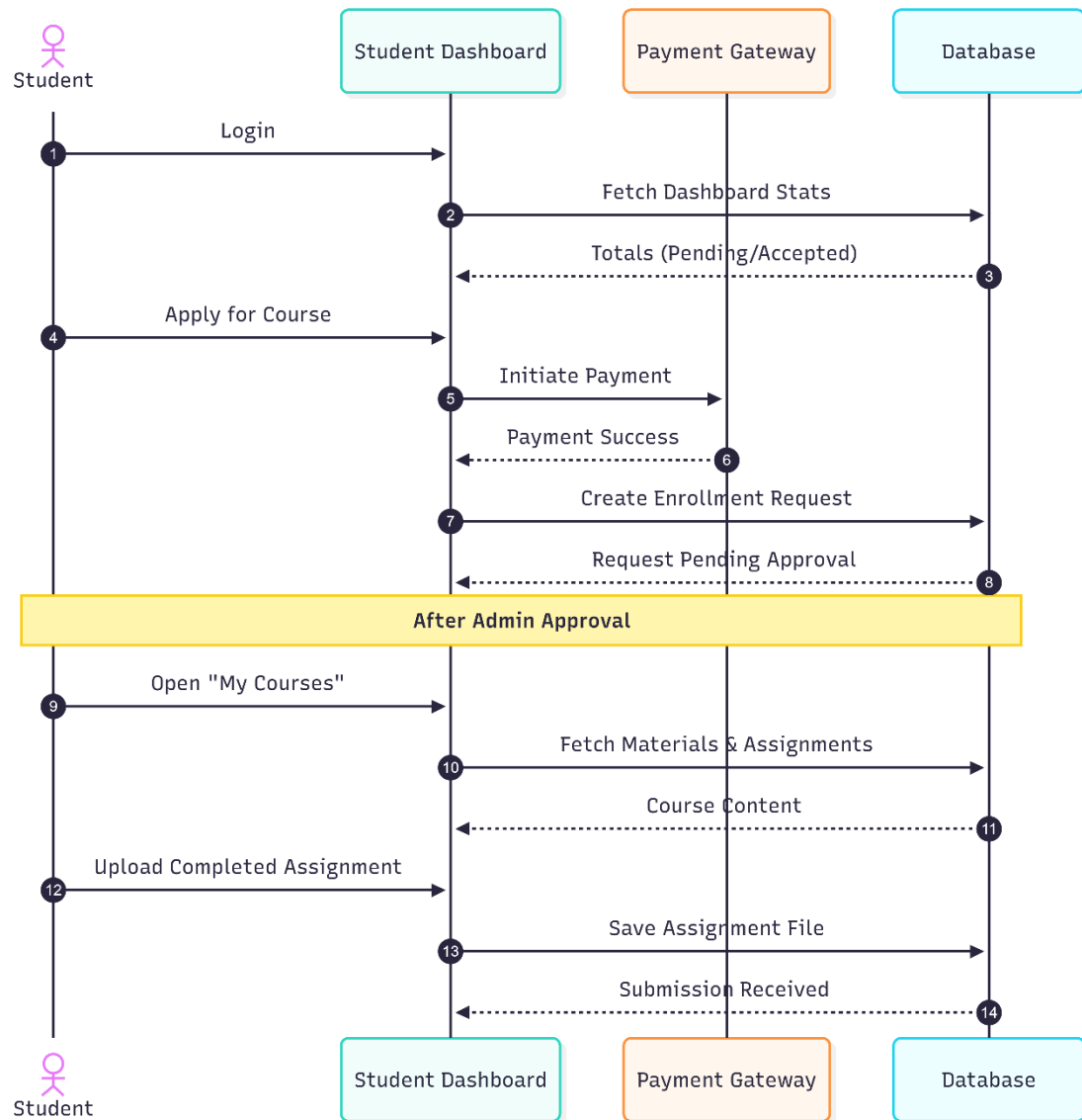


Figure 4.12 Sequence Diagram for Student

4.4 Class Diagram

A UML class diagram visually represents the structure of a system by showing its classes, attributes, methods, and the relationships between them. It helps everyone involved in a project like developers and designers and understand how the system is organized and how its components interact. It also helps to communicate and document the structure of the software.

4.4.1 Class Diagram for Coaching Center Management System(CCMS)

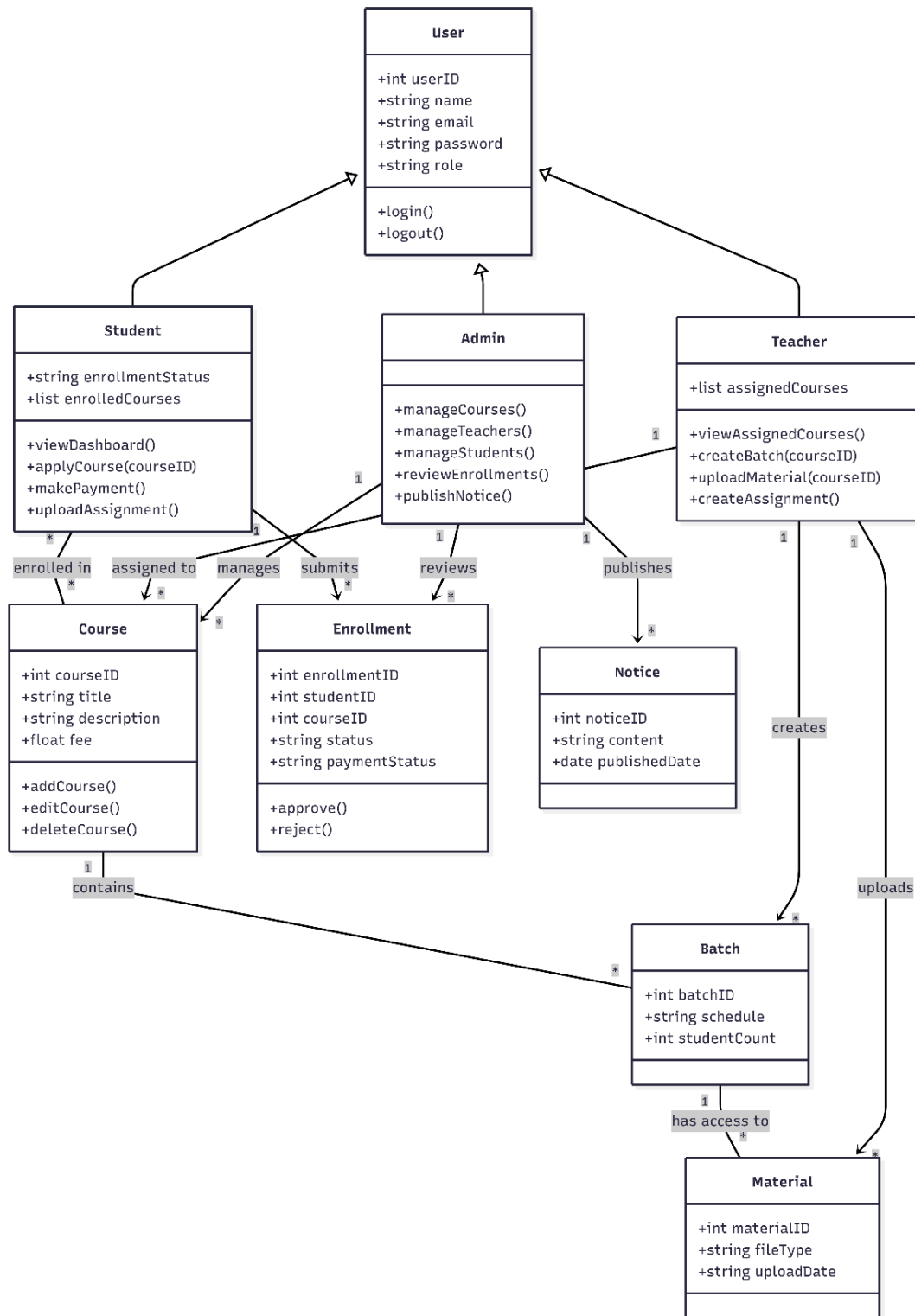


Figure 4.13 Class Diagram for Coaching Center Management System

Chapter 5.

Project Management

5.1 Risk Management Process

Risk Management Process is a structured approach used in software engineering and project management to identify, analyze, plan for and monitor risks that may affect the success of a project.

1. Risk identification: Identify project, product and business risks
2. Risk analysis: Assess the likelihood and consequences of these risks
3. Risk planning: Draw up plans to avoid or minimize the effects of the risk
4. Risk monitoring: Monitor the risks throughout the project

5.2 Risk Identification Table for Coaching Center Management System

Table 5.1: Risk Identification Table

ID	Risk Type	Risk Description
R01	Technology	Enrollment Bottleneck: Logic errors or database deadlocks in the approval module may leave students stuck in "Pending" status indefinitely.
R02	Technology	Upload Failure: Lack of file-size validation or storage handling for course materials may cause server crashes during large uploads.
R03	Data Integrity	Orphaned Data: Deleting a course without a "cascade" or "soft delete" strategy could crash the dashboards of students already enrolled in that course.
R04	Requirements	Authorization Leak: Insufficient server-side middleware could allow students to access restricted Admin routes via URL manipulation.
R05	People	Account Recovery: Manual account creation without a "Reset Password" feature turns the Admin into a constant help-desk for lost credentials.
R06	Process	Payment Discrepancy: Manual status updates are prone to human error, leading to disputes over "Paid" vs. "Not Paid" enrollment

		statuses.
R07	Tools	UI Incompatibility: Complex Admin tables and action buttons may become unresponsive or broken on older browsers or mobile devices.
R08	Estimation	Logic Complexity: Underestimating the time needed for "Batch" and "Schedule" logic may result in empty course pages at launch.

5.3 Risk Analysis Table for Coaching Center Management System

Table 5.2: Risk Analysis Table

ID	Risk Name	Probability	Effect	Justification
R01	Approval Bottleneck	Low (10-25%)	Serious	System failure at the approval stage halts the entire student user journey.
R02	Upload Failure	Moderate (25-50%)	Tolerable	Can be bypassed during demos by using smaller, optimized file types.
R03	Orphaned Data	Low (<10%)	Serious	Prevents critical runtime errors and "Null" exceptions on student dashboards.
R04	Auth Leak	Moderate (25-50%)	Serious	Security flaws in route protection significantly impact project credibility.
R05	Credential Loss	High (>50%)	Insignificant	Easily mitigated during presentations with standardized default passwords.
R06	Payment Error	Low (10-25%)	Tolerable	Human/logical errors can be corrected via direct database intervention if needed.
R07	Responsive UI Mess	Moderate (25-50%)	Tolerable	UI scaling issues can be managed through browser settings or desktop-

				first focus.
R08	Incomplete Materials	Moderate (25-50%)	Insignificant	Demonstration of the upload functionality is more critical than the volume of content.

5.4 Risk Planning Table for Coaching Center Management System

Table 5.3: Risk Planning Table

ID	Strategy Type	Action Plan	Implementation Strategy
R01	Contingency	Manual DB Override	Keep MongoDB Compass/Shell open to manually update enrollment status if the UI logic fails during the demo.
R02	Minimization	File Size Validation	Implement frontend constraints (e.g., < 5MB) to prevent server timeouts or storage crashes during uploads.
R03	Avoidance	Soft Delete Logic	Use an is_deleted flag rather than a hard delete to maintain data integrity for active student enrollments.
R04	Minimization	Middleware Guard	Apply Express.js middleware to verify user_role on the server-side for all protected administrative routes.
R05	Contingency	Super Admin Access	Hardcode a "Super Admin" credential in environment variables for emergency account recovery and testing.
R06	Minimization	Visual Badges	Use color-coded status badges (e.g., Red for Pending, Green for Paid) to provide immediate clarity on system state.

R07	Avoidance	Responsive Framework	Utilize Tailwind CSS's utility classes to ensure tables and dashboards remain functional across various screen resolutions.
R08	Minimization	Core-First Focus	Prioritize the development of the "Enrollment" loop over secondary features like "Notices" to ensure the main workflow is bug-free.

5.5 Risk Monitoring Table for Coaching Center Management System

Table 5.4: Risk Monitoring Table

Risk	Monitoring Trigger	Review Frequency
Approval Bottleneck	Perform end-to-end "Student-to-Admin" workflow test.	After any Database/Enrollment schema changes.
Upload Failure	Monitor Browser "Network" tab for 500/404 errors during file uploads.	Upon integration of new file-handling features.
Auth Leak	Attempt direct URL access to Admin routes from a Student account.	Weekly and after adding new backend routes.
Data Integrity	Delete a test course and verify stability of linked student dashboards.	After updates to the Course Management module.
Payment Logic	Verify instant "Paid/Not Paid" status updates in the Admin panel.	At the end of every development sprint.
UI/Responsive Mess	Toggle "Inspect Element" views (Desktop/Mobile) on the Dashboard.	After adding new tables or CSS modifications.
Teacher Account Loss	Validate login functionality for Admin-generated teacher credentials.	Monthly and 48 hours before the final presentation.
Unfinished Features	Benchmark current progress against "Must-Have" functional requirements.	Every Friday/End of the week.

5.6 The RMMM Plan for Coaching Center Management System

In Software Engineering, RMMM stands for Risk, Mitigation, Monitoring, and Management. It is a framework used in software project risk management to ensure that potential risks are identified, analyzed, and controlled throughout the software development life cycle.

Table 5.5: RMMM for R01

Risk Information Sheet – R01 (Enrollment Bottleneck)			
Risk ID: R01	Date: 8/12/2025	Prob: 15-20%	Impact: High
Description: The Admin's "Enrollment Approval" system fails to update the database, leaving students in a permanent "Pending" state even after payment.			
Refinement/context: Sub condition 1: Backend SQL/NoSQL update query contains logic errors. Sub condition 2: Payment status from the student side is not correctly linked to the admin's approval view.			
Mitigation/monitoring: • Implement automated unit tests for the enrollment status transition logic. • Add visual logs in the Admin Dashboard to confirm successful database writes.			
Management/contingency plan/trigger: Prepare a pre-written SQL script to manually toggle enrollment status to Approved' via the database console if the UI button fails during the demo. Trigger: Admin clicks "Approve" and the student dashboard status remains "Pending" after a page refresh.			
Current status: Database schema for enrollment status finalized; logic testing in progress.			
Originator: Developer		Assigned: Backend Developer	

Table 5.6: RMMM for R03

Risk Information Sheet – R03 (Orphaned Data)			
Risk ID: R03	Date: 8/12/2025	Prob: 10-15%	Impact: High
Description: Admin deletes a Teacher or Course, but the associated Student enrollment records remain, causing the system to crash when trying to fetch "missing" data.			
Refinement/context: sub condition 1: Missing "Cascade Delete" constraints in the database. sub condition 2: Frontend attempts to render course.name for a null object.			
Mitigation/monitoring: • Use "Soft Deletes" (e.g., is_active = false) instead of hard deletion. • Add null checks/Optional chaining in the frontend code.			
Management/contingency plan/trigger: Restore a database backup or manually recreate the deleted ID to stop the system from crashing during the live demo. Trigger: Student dashboard throws a "500 Error" or blank page after an Admin deletes a record.			
Current status: Reviewing database foreign key relationships.			
Originator: Developer		Assigned: Database Admin	

Table 5.7: RMMM for R04

Risk Information Sheet – R04 (Authorization Leak)			
Risk ID: R04	Date: 8/12/2025	Prob: 30-40%	Impact: High
Description: A Student gains access to Teacher or Admin dashboard features by manually typing URLs (e.g., /admin/teachers), leading to unauthorized data manipulation.			
Refinement/context: sub condition 1: Lack of server-side middleware to verify user roles for every route. sub condition 2: Client-side routing hides buttons but doesn't protect the underlying API endpoints.			
Mitigation/monitoring: • Implement a "Role-Based Access Control" (RBAC) middleware. • Use "Secret" tokens/sessions that expire after inactivity.			
Management/contingency plan/trigger: If a leak is discovered, immediately disable the "Delete" and "Block" functionality in the code until the routing logic is fixed. Trigger: A user with a 'Student' role successfully loads a 'Teacher' dashboard page.			
Current status: Basic login auth implemented; middleware role-check is under development.			
Originator: Developer		Assigned: Security Lead	

Table 5.8: RMMM for R06

Risk Information Sheet – R04 (Payment Discrepancy)			
Risk ID: R06	Date: 8/12/2025	Prob: 10-20%	Impact: Moderate
Description: Manual input errors lead to a student being marked as "Paid" without transaction verification or "Not Paid" despite having completed the payment.			
Refinement/context: sub condition 1: The Admin dashboard lacks a field to input a transaction reference ID. sub condition 2: No automated confirmation notification is sent to the student after payment status is updated.			
Mitigation/monitoring: • Add a TransactionID field to the enrollment request schema. • Audit 10% of enrollment records weekly against the physical payment logs.			
Management/contingency plan/trigger: Implement an "Update Log" to track which admin made changes to a student's payment status. Trigger: More than two reported payment disputes from students within a single week.			
Current status: UI updated to show "Payment Reference" column; backend validation in progress.			
Originator: Developer		Assigned: Admin/QA	

Chapter 6.

Project Planning

6.1 Project Planning

For any task, a full estimate is essential before starting. It entails estimating the projects completion date, rating the projects viability, rating the extent of the work, and identifying the resources required. Tasks are broken down; the specifics of each phase are clarified and the funds required for effective execution are measured through estimation. For realistic plan and deadline retention, estimating the necessary timetable is crucial. In order assess project viability within limitations like money, technology, skill levels, and possible dangers, estimation also entails qualitative analysis. This logical approach lays the foundation for efficient project management and operation.

6.2 Project Estimation

A software project's estimate accuracy depends on a number of important factors:

1. Accurate Product Size Estimation: An essential starting point involves precisely

gauging the size of the intended product. This estimation forms the cornerstone for subsequent evaluations and planning within the project.

2. Translation into Effort, Time and Costs: Converting the estimated product size into tangible aspects such as human effort, calendar time, and financial resources is pivotal. This conversion allows for a realistic projection of the resources needed for successful project completion.

3. Alignment with Team Capabilities: The effectiveness of the project plan depends

on how well it aligns with the skills and expertise of the software team. Crafting a plan that harnesses the team's strengths increases the likelihood of achieving project

objectives.

4. Consistency in Requirements and Environment: The stability of product requirements and the work environment are crucial. Consistent requirements and a supportive work environment enhances the predictability and stability of the software engineering effort.

6.3 Function Oriented Matrices

Instead of focusing on software values, function point-based estimate concentrates on information domain values. Five information domain properties are compared in order to derive function points.

1. Data Functions

- a) Internal Logical Files
- b) External Interface Files

2. Transaction Functions

- a) External Inputs
- b) External Outputs
- c) External Inquiries

3. Count of External Inputs: Each user input that provides distinct application-oriented data to the software is counted. Input should be differentiated from inquiries.

4. Count of External Outputs: Each user output that provides application-oriented information to the user is counted.

5. Count of External Inquiries: An inquiry is defined as an online input that results in the generation of immediate software responses in the form of online outputs. Each distinct inquiry is counted.

6.4 Function Point Calculation

Here is the Functionality, Input, and Output Table for Coaching Center Management System, organized by the main user roles (Admin, Teacher, and Student).

Table 6.1: Functionality, input and output table

Functionality	Input	Output
Admin		
Admin can add, modify, or assign teachers to courses	Teacher Name, Email, Password, Designation, Course ID to assign	Confirmation message for account/course update or an error message
Admin can manage enrollment requests	Select Enrollment ID, Click Approve or Reject	Update student status to 'Approved' or 'Rejected' and display confirmation
Admin can publish system-wide notices	Notice Title, Content, Target Audience	Notice becomes visible on the public page and all user dashboards
Admin can monitor messages from the public	Click on Messages tab	List of messages including Sender Name, Email, and Message body
Teacher		
Teacher can manage course batches	Batch Number, Schedule, Course Selection	Confirmation of batch creation or an error message
Teacher can upload course materials	Material Title, File (PDF/PPT), Batch ID	Material appears in the student dashboard for that specific batch
Teacher can create assignments	Assignment Title, Description, Due Date, Course ID	List of assignments displayed to all enrolled students in that course
Student		
Student can register and apply for courses	Name, Email, Password, Selected Course ID	Account created; enrollment status displayed as 'Pending'
Student can pay via SSLCOMMERZ	Click Pay, Card/Mobile Banking details (on SSL page)	Payment status updated to 'Paid' and SSL transaction ID stored
Student can upload assignments	Select Assignment, Choose File to upload	Confirmation of submission and timestamp displayed to student
Student can view course progress	Click on My Courses	List of batch numbers, schedules, and materials for enrolled courses

6.4.1 Identifying Complexity of Transaction Function

Table 6.2: Complexity Table for Transaction Functions

Transaction Functions	Fields / File Involvement	FTRs	DETs
Student Registration (EI)	Fields: Name, Email, Password, Role, Submit, Confirm/Error Msg. File: Users	1	6
Login Authentication (EQ)	Fields: Email, Password, Submit, Auth Token/Error Msg. File: Users	1	4
Create Teacher Account (EI)	Fields: Name, Email, Password, Designation, Bio, Submit, Success Msg. Files: Users, Teachers	2	7
Create Course (EI)	Fields: Title, Description, Price, Assigned Teacher, Submit, Success/Error Msg. Files: Courses, Teachers	2	6
Enrollment Request (EI)	Fields: Student ID, Course ID, Payment Status, Apply Button, Confirmation. File: Enrollments	1	5
Approve Enrollment (EI)	Fields: Enrollment ID, Batch ID, Update Status (Approved), Submit. Files: Enrollments, Batches	2	4
Upload Material (EI)	Fields: Title, File Path, Course ID, Batch ID, Submit, Success Msg. Files: Materials, Batches	2	6
Create Notice (EI)	Fields: Title, Content, Target Audience, Submit, Confirm Msg. File: Notices	1	5
View Student List (EQ)	Fields: Input: Course ID. Output: Name, Email, Batch, Enrollment Status. Files: Users, Enrollments	2	5
Admin Overview (EO)	Fields: Output: Total Students, Total Teachers, Total Courses (Calculated counts). Files: Users, Courses	2	3
Teacher Dashboard (EO)	Fields: Output: Assigned Courses Count, Total Materials, Total Students. Files: Courses, Materials, Enrollments	3	3
Public Contact Message (EI)	Fields: Name, Email, Subject, Message, Submit. File: Messages	1	5

6.4.2 Identifying Complexity of Data Function

Table 6.3: Complexity Table for Data Functions

Data Functions	Fields / File Involvement	RETs	DETs
Users (ILF)	Fields: _id, name, email, password, role, status, joinDate. Sub-group: designation, bio, specialization.	2	10
Courses (ILF)	Fields: _id, title, description, price, teacherId, adminId, dateCreated. Sub-groups: batches, materials.	3	14
Enrollments (ILF)	Fields: _id, studentId, courseId, batchId, paymentStatus, enrollmentStatus, ssl_tran_id, ssl_session_key, appliedDate.	1	9
Payment Gateway (EIF)	Data Referenced from SSLCOMMERZ: Transaction ID, Store ID, Currency, Amount, Card Type, Validation Status, Risk Title.	1	7
Assignments (ILF)	Fields: _id, title, description, dueDate, courseId. Sub-group: studentId, filePath, submittedAt.	2	8
Notices (ILF)	Fields: _id, title, content, targetAudience, createdBy, date.	1	6
Messages (ILF)	Fields: _id, senderName, senderEmail, subject, message, receivedAt.	1	6

6.4.3 Unadjusted Function Point Contribution for Transaction Functions

EI Table

FTR'S	DATA ELEMENTS		
	1-4	5-15	>15
0-1	Low	Low	Average
2	Low	Average	High
3 or more	Average	High	High

EO and EQ Table

FTR'S	DATA ELEMENTS		
	1-5	6-19	>19
0-1	Low	Low	Average
2-3	Low	Average	High
>3	Average	High	High

Values for transactions

RATING	DATA ELEMENTS		
	EO	EQ	EI
Low	4	3	3
Average	5	4	4
High	7	6	6

Figure 6.1 Formula Table for EI and EO value

Table 6.4: Unadjusted Function Point Contribution for Transaction Functions

Transaction Functions	Fields / File Involvement	FTRs	DETs	Complexity	UFP
Student Registration (EI)	Fields: Name, Email, Password, Role, Submit, Confirm/Error Msg. File: Users	1	6	Low	3
Login Authentication (EQ)	Fields: Email, Password, Submit, Auth Token/Error Msg. File: Users	1	4	Low	3
Create Teacher Account (EI)	Fields: Name, Email, Password, Designation, Bio, Submit, Success Msg. Files: Users, Teachers	2	7	Average	4

Create Course (EI)	Fields: Title, Description, Price, Assigned Teacher, Submit, Success/Error Msg. Files: Courses, Teachers	2	6	Average	4
Enrollment Request (EI)	Fields: Student ID, Course ID, Payment Status, Apply Button, Confirmation. File: Enrollments	1	5	Low	3
Approve Enrollment (EI)	Fields: Enrollment ID, Batch ID, Update Status (Approved), Submit. Files: Enrollments, Batches	2	4	Low	3
Upload Material (EI)	Fields: Title, File Path, Course ID, Batch ID, Submit, Success Msg. Files: Materials, Batches	2	6	Average	4
Create Notice (EI)	Fields: Title, Content, Target Audience, Submit, Confirm Msg. File: Notices	1	5	Low	3
View Student List (EQ)	Fields: Input: Course ID. Output: Name, Email, Batch, Enrollment Status. Files: Users, Enrollments	2	5	Low	3
Admin Overview (EO)	Fields: Output: Total Students, Total Teachers, Total Courses (Calculated counts). Files: Users, Courses	2	3	Low	4
Teacher Dashboard (EO)	Fields: Output: Assigned Courses Count, Total Materials, Total Students. Files: Courses, Materials, Enrollments	3	3	Low	4
Public Contact Message (EI)	Fields: Name, Email, Subject, Message, Submit. File: Messages	1	5	Low	3
Total					41

6.4.4 Unadjusted Function Point Contribution for Data Functions

RET'S	DATA ELEMENTS		
	1-19	20-50	>50
1	Low	Low	Average
2-5	Low	Average	High
>5	Average	High	High

Values for transactions

RATING	DATA ELEMENTS	
	ILF	EIF
Low	7	5
Average	10	7
High	15	10

Figure 6.2 Formula Table for ILF and EIF value

Table 6.5: Unadjusted Function Point Contribution for Data Functions

Data Functions	Fields / File Involvement	RETs	DETs	Complexity	UFP
Users (ILF)	Fields: _id, name, email, password, role, status, joinDate. Sub-group: designation, bio, specialization.	2	10	Low	7
Courses (ILF)	Fields: _id, title, description, price, teacherId, adminId, dateCreated. Sub-groups: batches, materials.	3	14	Low	7
Enrollments (ILF)	Fields: _id, studentId, courseId, batchId, paymentStatus, enrollmentStatus, ssl_tran_id,	1	9	Low	7

	ssl_session_key, appliedDate.				
Payment Gateway (EIF)	Data Referenced from SSLCOMMERZ: Transaction ID, Store ID, Currency, Amount, Card Type, Validation Status, Risk Title.	1	7	Low	5
Assignments (ILF)	Fields: _id, title, description, dueDate, courseId. Sub-group: studentId, filePath, submittedAt.	2	8	Low	7
Notices (ILF)	Fields: _id, title, content, targetAudience, createdBy, date.	1	6	Low	7
Messages (ILF)	Fields: _id, senderName, senderEmail, subject, message, receivedAt.	1	6	Low	7
Total					47

6.4.5 Performance and Environmental Impact

Table 6.6: Total Degree of Influence (TDI) Calculation

General System Characteristic	DI
Data communications	3
Distributed data processing	2
Performance	3
Heavily used configuration	1
Transaction rate	3
On-Line data entry	3
End-user efficiency	3
On-Line update	2

Complex processing	2
Reusability	2
Installation ease	3
Operational ease	3
Multiple sites	0
Facilitate change	2
Total Degree of Influence (TDI)	32

$$\text{Value Adjustment Factor (VAF)} = 0.65 + (0.01 \times \text{TDI}) = 0.65 + (0.01 \times 32) = 0.97$$

6.4.6 Counting Adjusted Function Point

$$\text{UFP} = \text{UFP (TF)} + \text{UFP (DF)} = 41 + 47 = 88$$

$$\text{Adjusted Function Point Estimation (AFP)} = \text{UFP} * \text{VAF} = 88 * 0.97 = 85.36$$

$$\text{Effort for Project} = \text{AFP} * \text{Productivity for JavaScript}$$

$$= 85.36 * 8 = 682.88 \text{ person hours}$$

$$= 682.88 / 8 = 85.36 \text{ person days (8 hours per day)}$$

$$= 85.36 / 24 = 3.56 \text{ Man months (24 days per month)}$$

Approx. 3.56 months need for developing the project

6.5 Project Schedule Chart

In this project scheduling chart, time is measured in weeks over a total of twelve weeks.

Table 6.7: Weekly Breakdown of Project

Week	Phase	Key Tasks
W1-W2	Planning & Design	UI/UX wireframes; Database Schema; Repo setup.
W3	Public & Auth	Landing pages; Registration & Login implementation.
W4-W5	Admin Dashboard	Teacher Management (CRUD); Course Management; Notices.
W6	Student Core	Enrollment logic; Dashboard stats; Payment placeholders.
W7-W8	Teacher Dashboard	Batch/Schedule creation; Material Uploads; Assignments.
W9	Enrollment Logic	Admin Approval workflow; Paid/Not Paid integration.
W10	Student Features	Assignment uploads; Enrollment status tracking.
W11	Testing & QA	Security audits (RBAC); Bug fixing; Cross-browser checks.
W12	Final Delivery	Documentation; Final Demo; System Presentation.

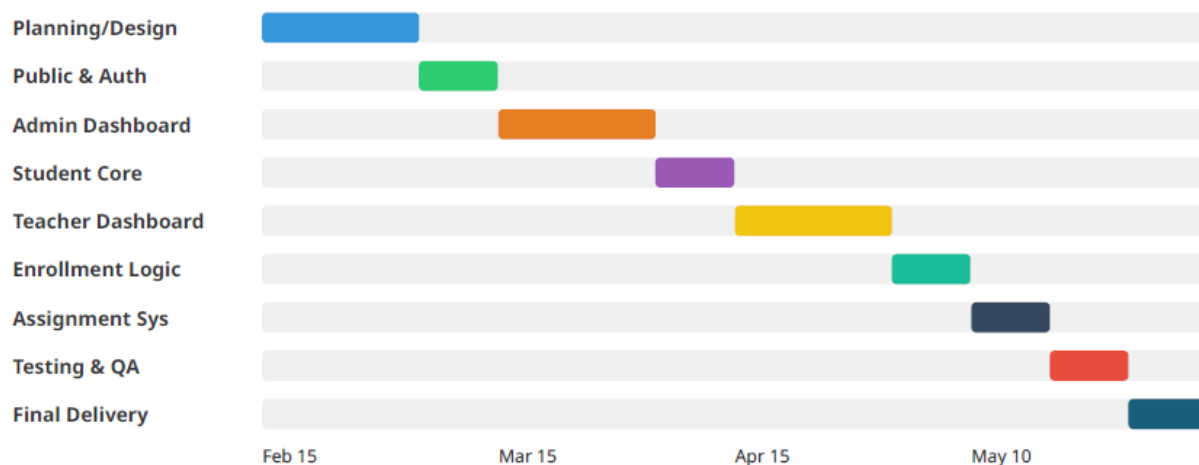


Figure 6.3: Project Schedule Chart

6.6 Project Cost Estimation

Project estimation primarily focuses on forecasting the expenses involved in developing software. For my project, the estimation includes the value of time dedicated to development, infrastructure costs, software tools, licenses or frameworks required and other miscellaneous expenses. Even as a solo project, these figures represent the professional value and resources utilized during the 12-week development lifecycle.

6.6.1 Personnel Cost

Table 6.8: Personnel cost estimation

Role	Estimated hours	Rate (BDT per hour)	Total wage (Tk)
Designer	132	200	26,400
Developer	350	250	87,500
Tester	130	150	19,500
Total			133,400

6.6.2 Hardware Cost

Table 6.9: Hardware cost estimation

Hardware	Depreciation Calculation	Expense (Tk)
Development Laptop	85,000 * 20%	17,000
Networking Setup	5,000 * 20%	1,000
Total		18,000 Tk

6.6.3 Software Cost

Table 6.10: Software cost estimation

Software	Cost
Development Tools (VS Code)	Open source
Frameworks (MERN Stack)	Open source
Database (MongoDB Atlas)	Open source
Total	0 Tk

6.6.4 Other Cost

Table 6.11: Other cost estimation

Item	Cost (Tk)
Internet	5000
Electricity	1000
Others	1000
Total	7000

6.6.5 Accounts Table

Table 6.12: Accounts Table

Cost Category	Amount (Tk)
Personnel cost	133,400
Hardware cost	18,000
Software cost	0
Other cost	7000
Total	158,400

Chapter 7.

Designing

7.1 Dataflow Diagram

A data flow diagram (DFD) is a graphical representation of how data moves through a system. It shows the flow of information between processes, data stores, external entities and data inputs/outputs. DFDs help in understanding the flow of data, identifying bottlenecks and ensuring efficient system design. They are commonly used in system analysis and design to visualize how a system processes data without detailing technical specifics.

7.1.1 Context level DFD

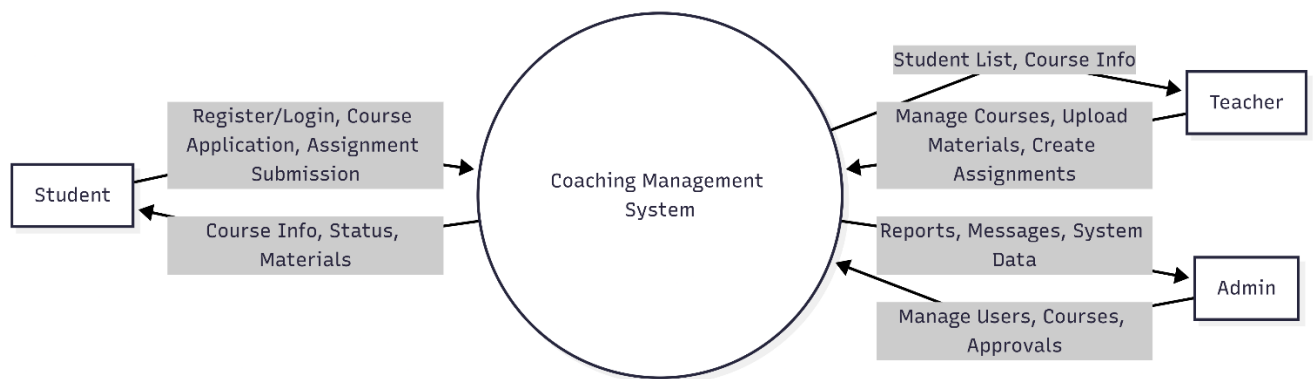


Figure 7.1: Context level / Level 0 DFD

7.1.2 Level 1 DFD

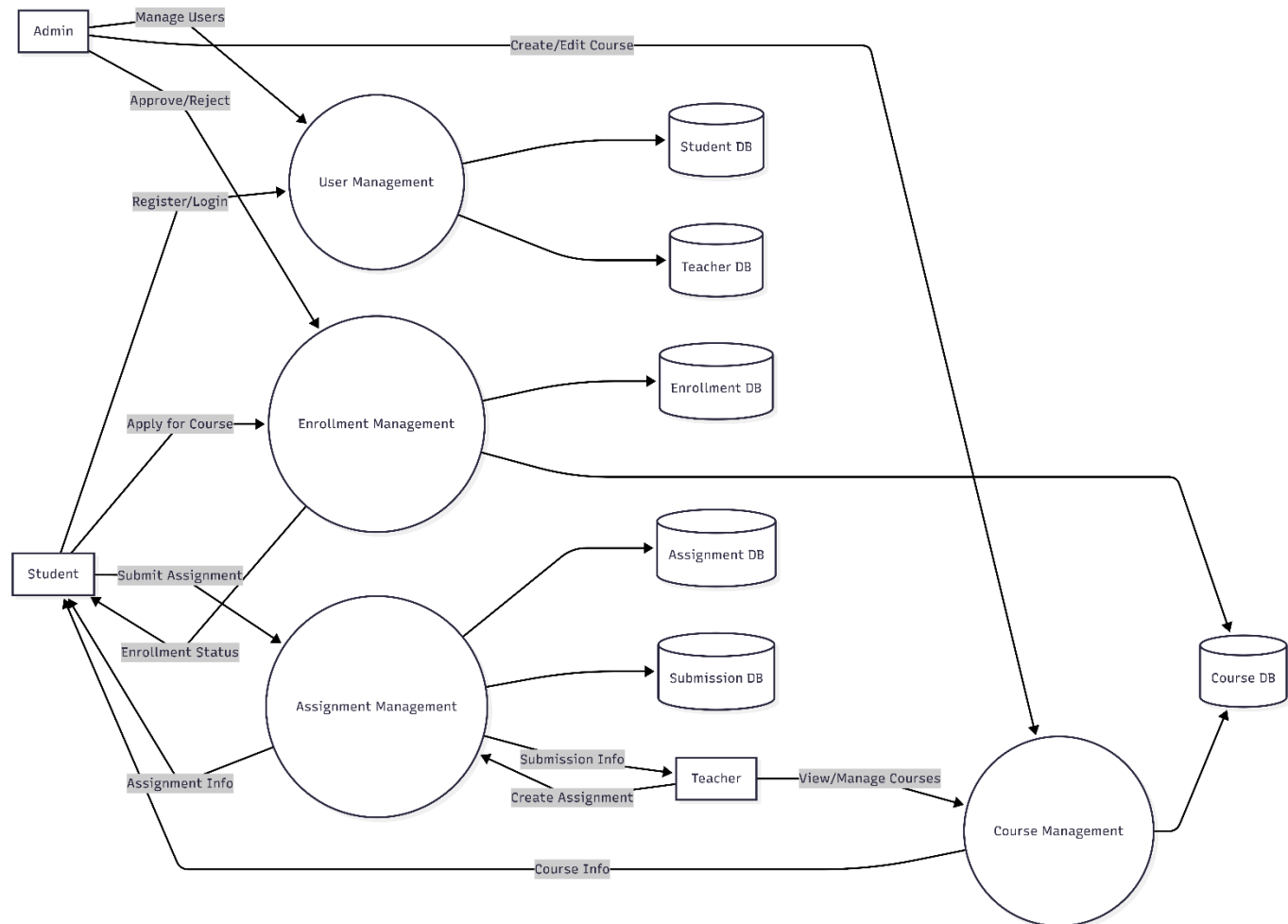


Figure 7.2: Level 1 DFD

7.1.3 Level 2 DFD

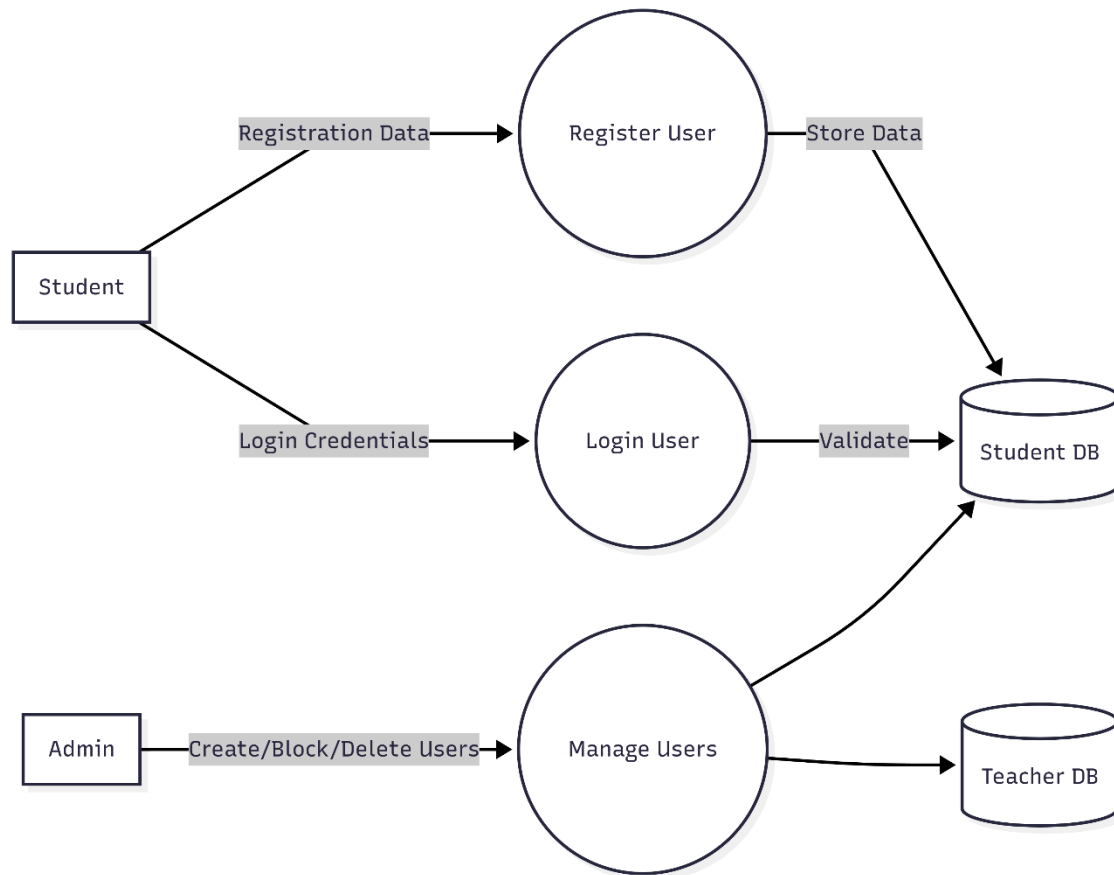


Figure 7.3: Level 2 DFD of User Management Process

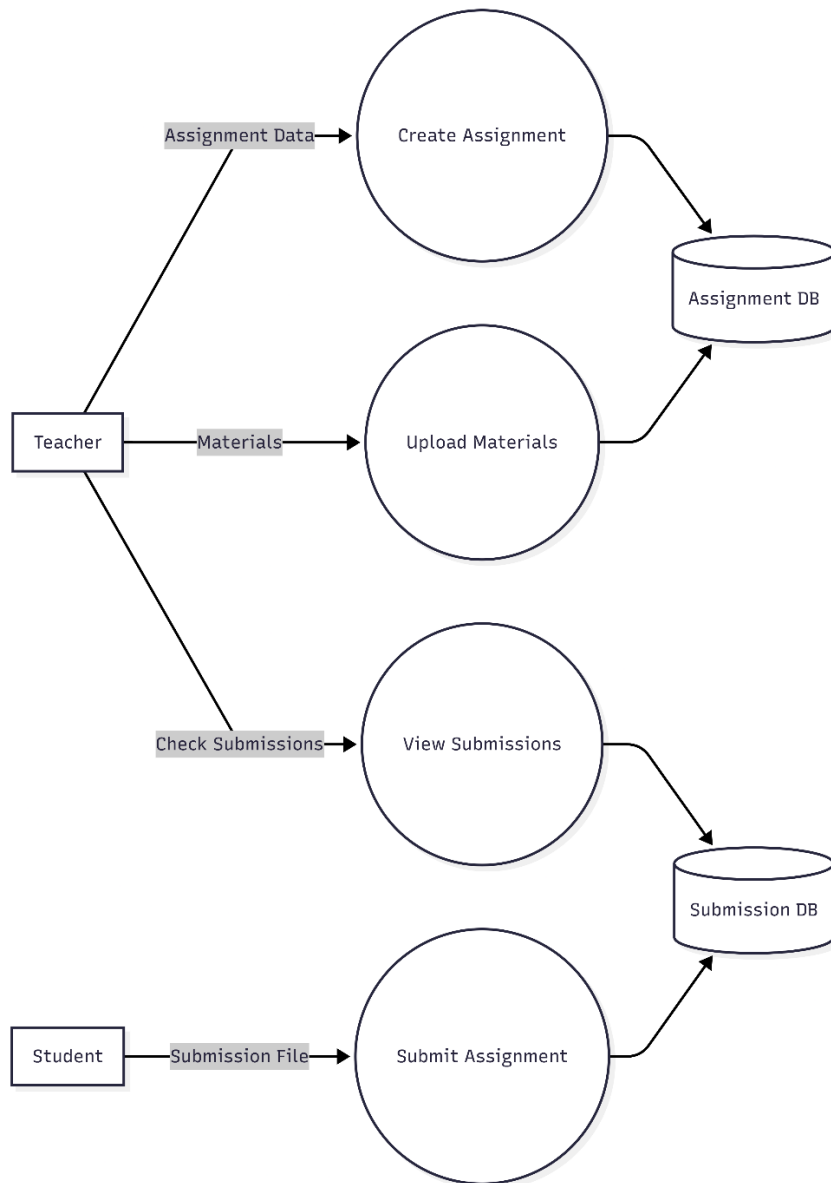


Figure 7.4: Level 2 DFD of Assignment Process

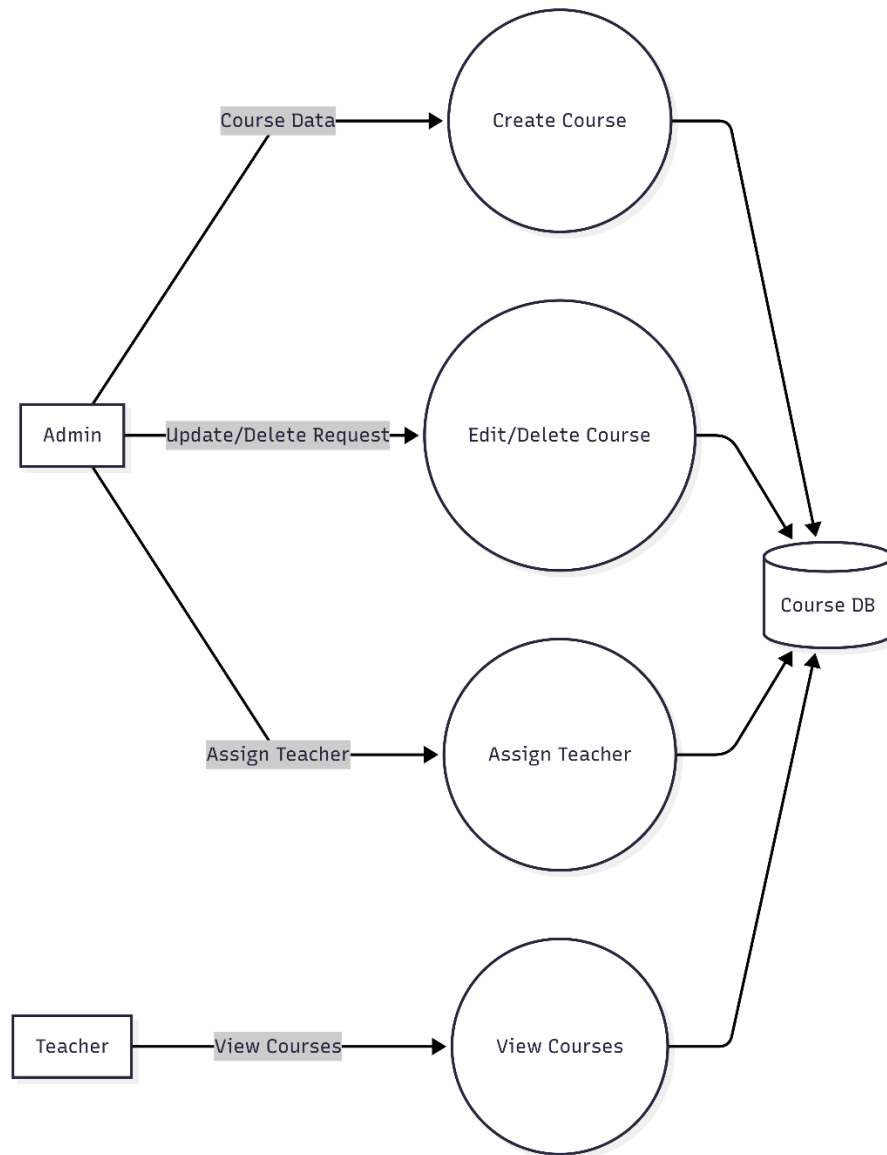


Figure 7.5: Level 2 DFD of Course Management Process

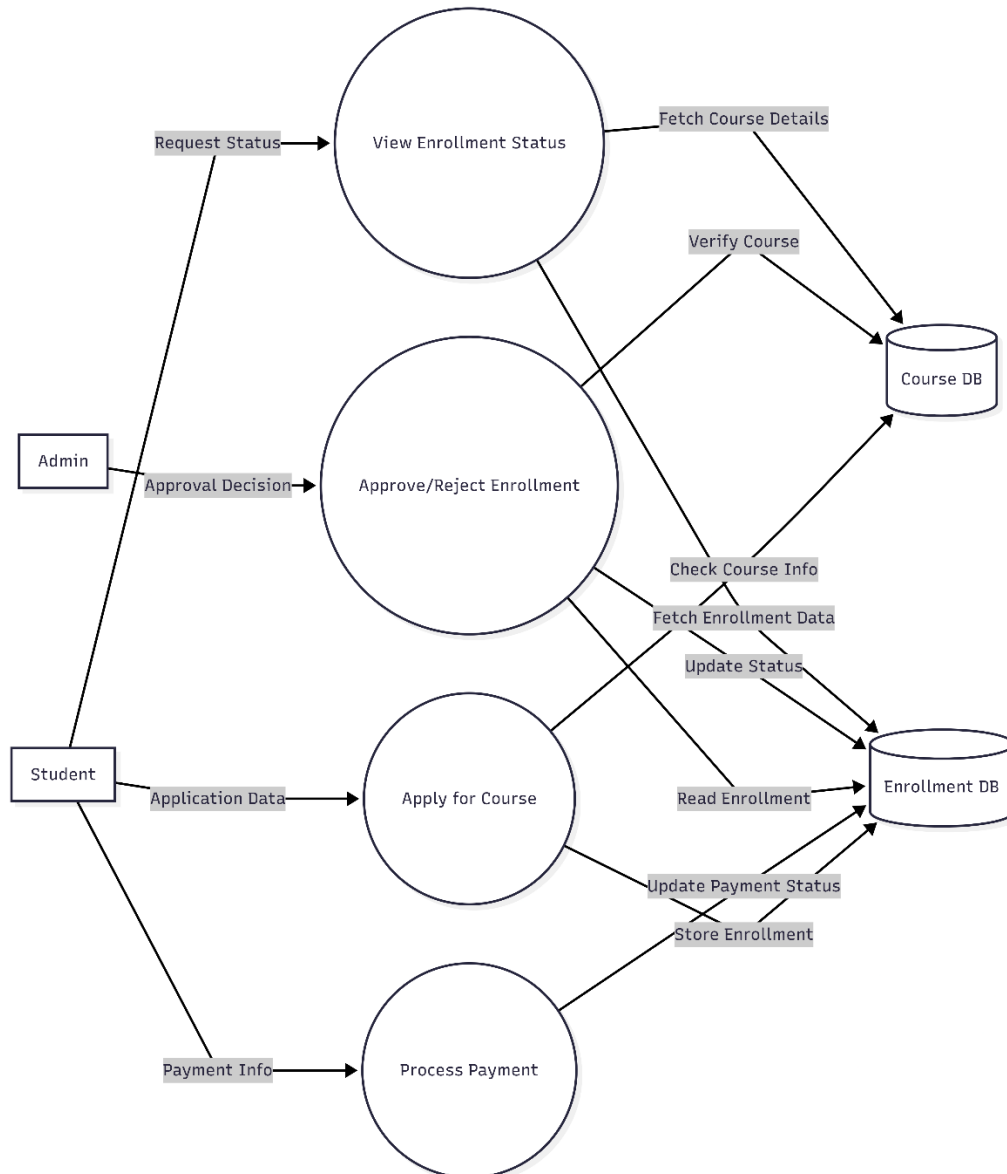


Figure 7.6: Level 2 DFD of Course Enrollment Process

7.2 Entity Relationship Diagram

An Entity-Relationship (ER) diagram is a visual representation of relationships between entities in a database. It depicts entities (such as people, objects, or concepts) as rectangles and their relationships as diamonds or lines, with attributes of each entity shown as ovals. ER diagrams help design and organize database structures by illustrating how entities interact

with one another, defining key relationships like one-to-one, one-to-many, or many-to-many. They are commonly used in database design to ensure data is structured efficiently.

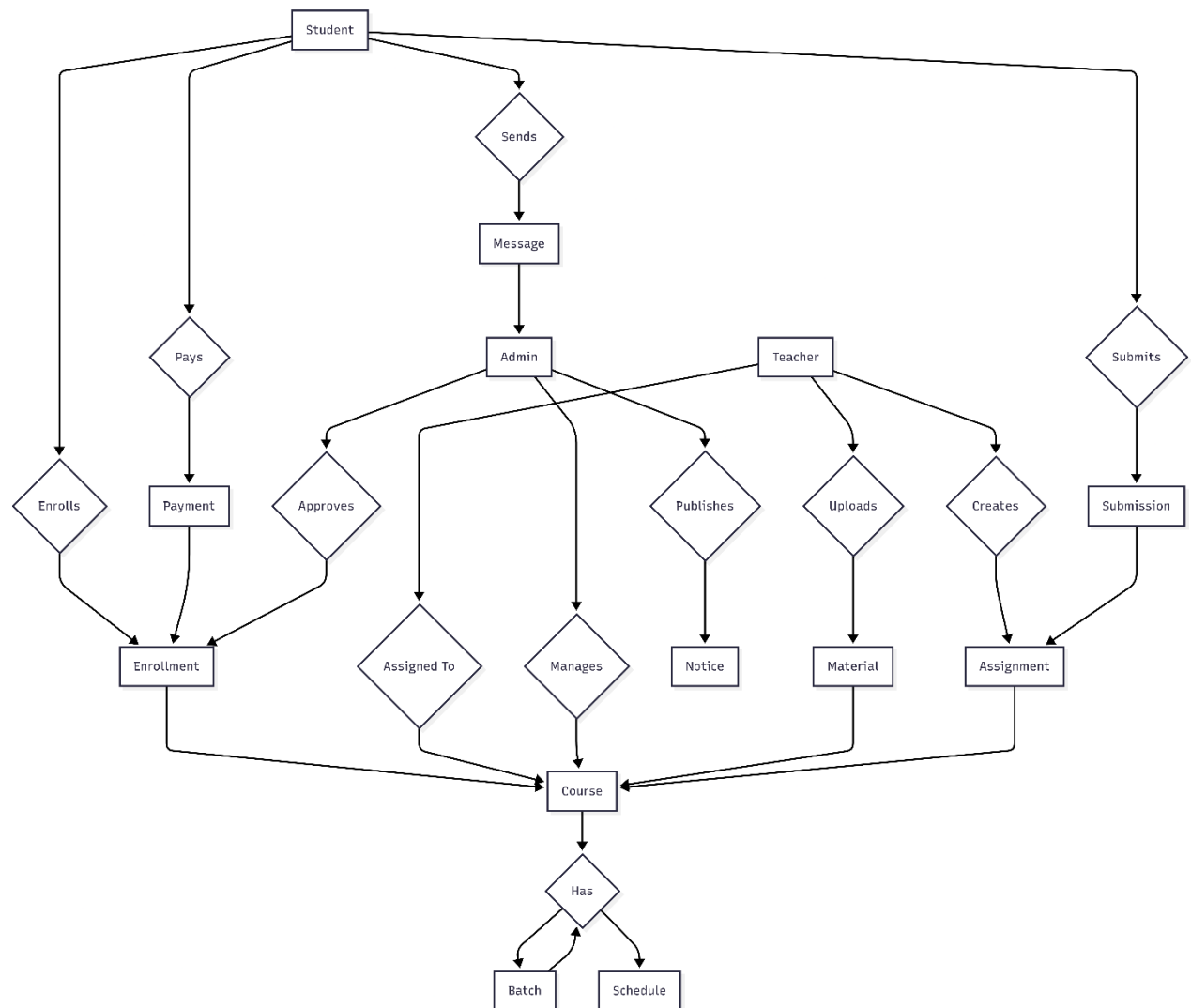


Figure 7.7: ER Diagram for Coaching Center Management System

7.3 Database Table Structure

Coaching Center Management System database table structure.

1. Users Table

This table handles authentication for all three roles: Admin, Teacher, and Student.

- user_id (Primary Key)
- name
- email (Unique)
- password
- role (Enum: 'Admin', 'Teacher', 'Student')
- status (Enum: 'Active', 'Blocked')
- created_at

2. Teachers Table

Extends user data for teacher-specific profiles.

- teacher_id (Primary Key, Foreign Key to Users.user_id)
- designation
- bio
- specialization

3. Courses Table

Stores course details and links them to the assigned teacher.

- course_id (Primary Key)
- course_name
- description
- price

- assigned_teacher_id (Foreign Key to Teachers.teacher_id)
- created_by_admin_id (Foreign Key to Users.user_id)

4. Batches Table

Teachers create batches for their assigned courses.

- batch_id (Primary Key)
- batch_number
- schedule (e.g., "Mon/Wed 10:00 AM")
- course_id (Foreign Key to Courses.course_id)

5. Enrollments Table

Manages the relationship between students and courses, including the approval workflow.

- enrollment_id (Primary Key)
- student_id (Foreign Key to Users.user_id)
- course_id (Foreign Key to Courses.course_id)
- batch_id (Foreign Key to Batches.batch_id, Nullable until approved)
- payment_status (Enum: 'Paid', 'Not Paid')
- enrollment_status (Enum: 'Pending', 'Approved', 'Rejected')
- applied_at

6. Materials Table

Uploaded by teachers for specific batches/courses.

- material_id (Primary Key)
- title
- file_path (Storage URL)
- course_id (Foreign Key to Courses.course_id)

- batch_id (Foreign Key to Batches.batch_id)
- uploaded_by_teacher_id (Foreign Key to Teachers.teacher_id)

7. Assignments Table

Teachers create tasks, and students upload their work.

- assignment_id (Primary Key)
- title
- description
- due_date
- course_id (Foreign Key to Courses.course_id)

8. Assignment_Submissions Table

- submission_id (Primary Key)
- assignment_id (Foreign Key to Assignments.assignment_id)
- student_id (Foreign Key to Users.user_id)
- file_path (Submission URL)
- submitted_at

9. General Communications (Notices & Messages)

- Notices: notice_id, title, content, created_at (Visible to all).
- Messages: message_id, sender_name, sender_email, subject, content, received_at (Public to Admin).

This normalized structure ensures that when an Admin approves an enrollment, they can assign a student to a specific batch_id, which then automatically gives the student access to that batch's schedule and materials.

7.4 Interface Design

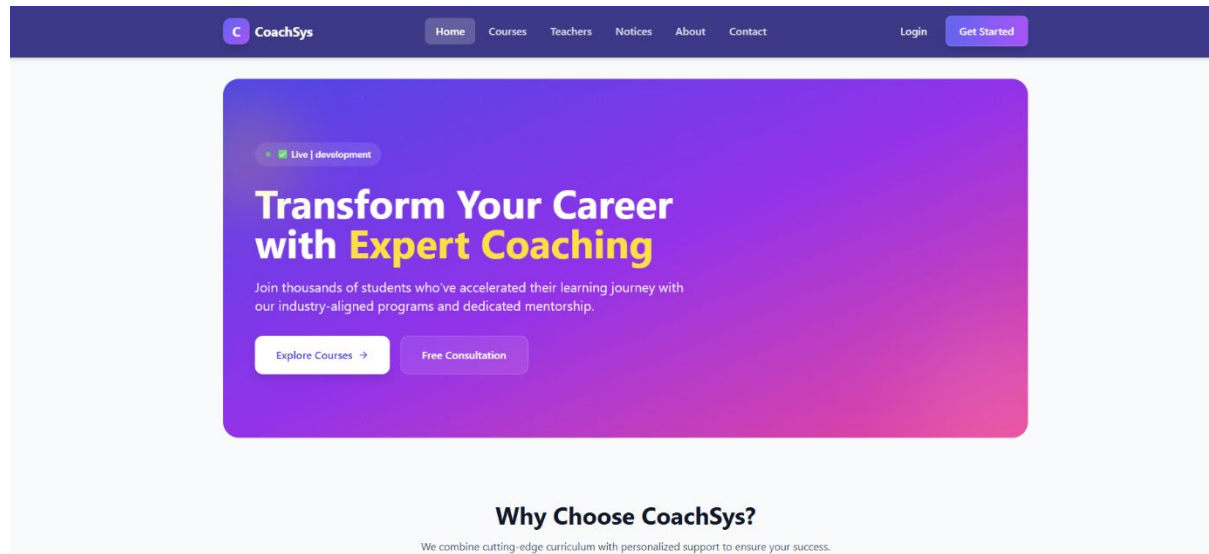


Figure 7.8: Homepage

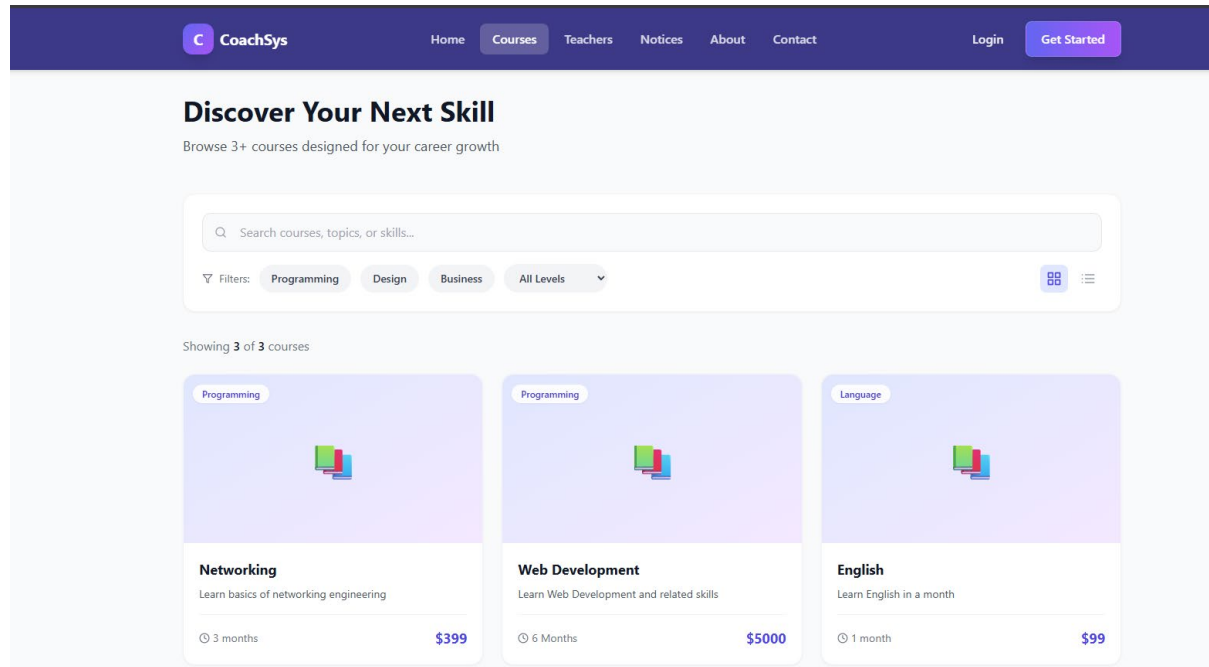


Figure 7.9: Courses Page

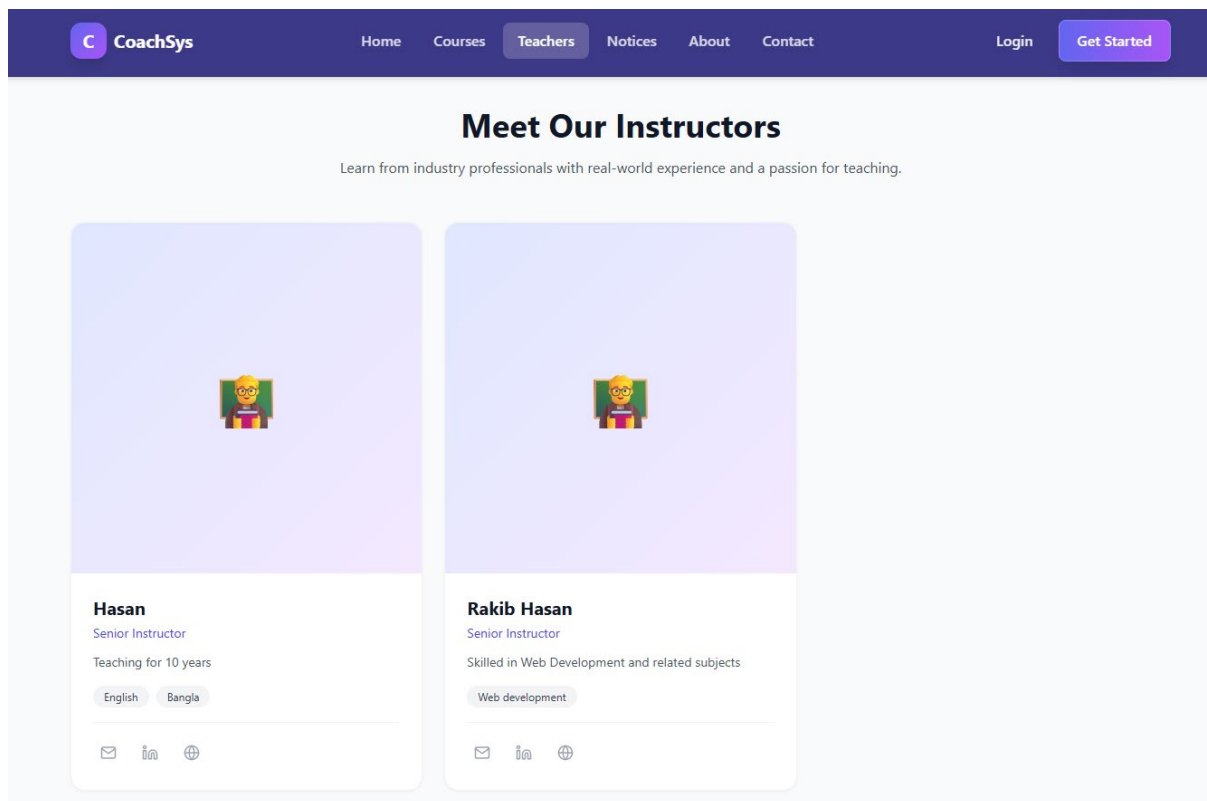


Figure 7.10: Teacher Page

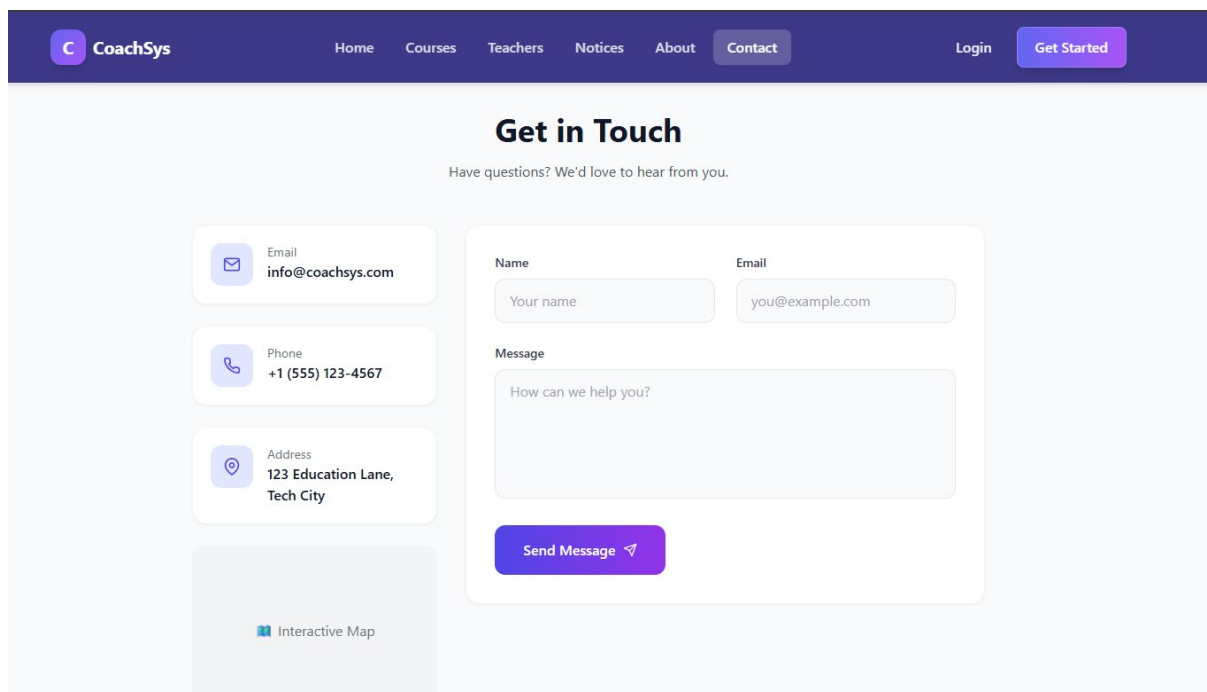
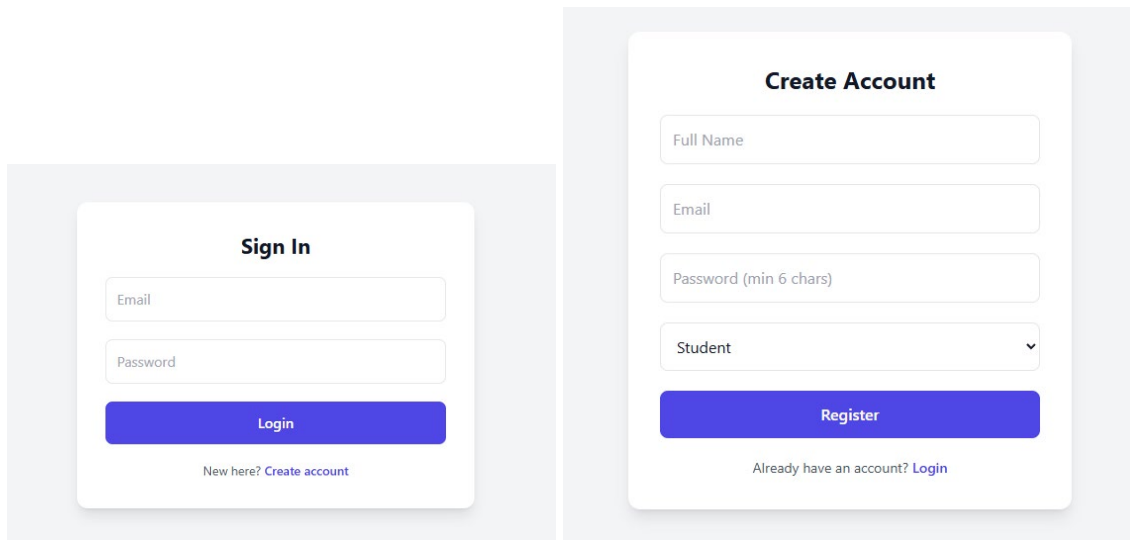
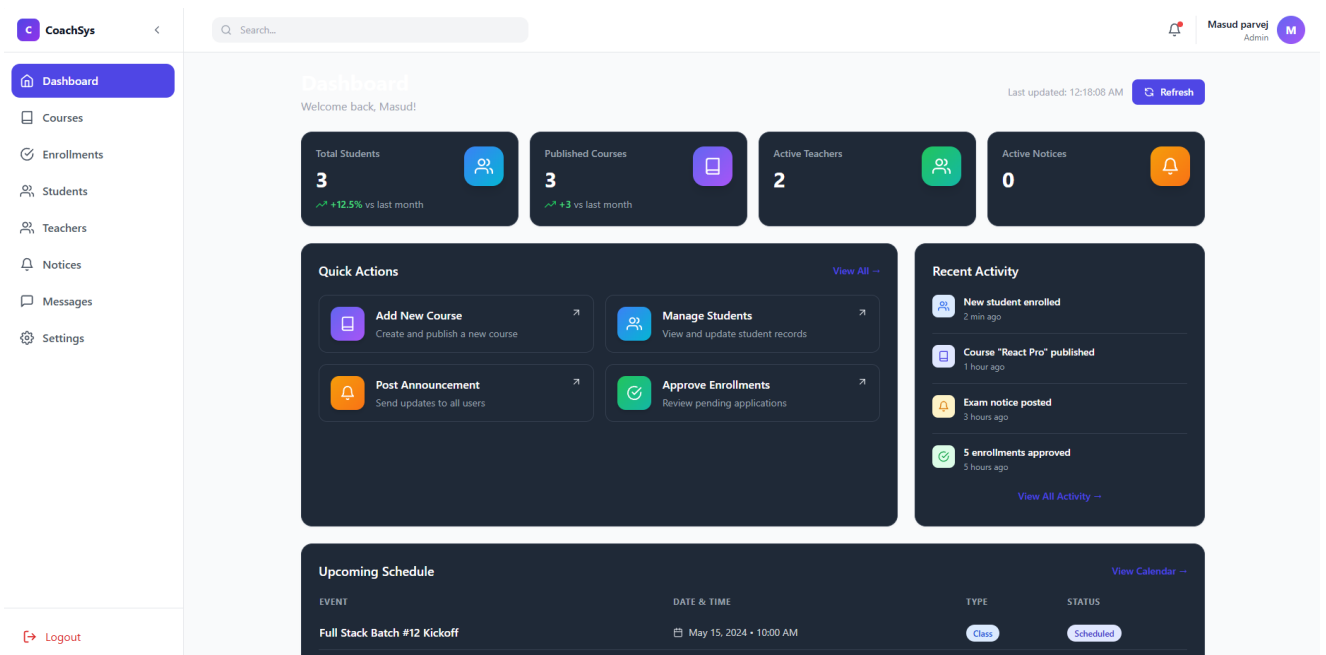


Figure 7.11: Contact Page



The image displays two side-by-side login and registration forms. The left form, titled "Sign In", features input fields for "Email" and "Password", a blue "Login" button, and a link "New here? Create account". The right form, titled "Create Account", includes input fields for "Full Name", "Email", and "Password (min 6 chars)", a dropdown menu for "Student", a blue "Register" button, and a link "Already have an account? Login".

Figure 7.12: Login and Register



The image shows the Admin Dashboard of the CoachSys application. The dashboard includes a sidebar with navigation links: Dashboard, Courses, Enrollments, Students, Teachers, Notices, Messages, and Settings. The main content area displays a "Dashboard" header with a welcome message and a "Refresh" button. Below the header, there are four summary cards: Total Students (3, +12.5% vs last month), Published Courses (3, +3 vs last month), Active Teachers (2), and Active Notices (0). The "Quick Actions" section contains four cards: Add New Course, Manage Students, Post Announcement, and Approve Enrollments. The "Recent Activity" section lists four events: New student enrolled, Course "React Pro" published, Exam notice posted, and 5 enrollments approved. The "Upcoming Schedule" section shows a table with columns for Event, Date & Time, Type, and Status, with one entry: Full Stack Batch #12 Kickoff on May 15, 2024 at 10:00 AM, Type: Class, Status: Scheduled.

EVENT	DATE & TIME	TYPE	STATUS
Full Stack Batch #12 Kickoff	May 15, 2024 • 10:00 AM	Class	Scheduled

Figure 7.13: Admin Dashboard

CoachSys < Search...

Course Management
Create, edit, publish and assign teachers to courses. [+ Add course](#)

Search title, slug, category... All Refresh

TITLE	STATUS	TEACHER	FEE	CATEGORY	LEVEL	CREATED	ACTIONS
Networking /networking	published	A. Hasan	399	Programming	Beginner	5/6/2026	Edit Delete
Web Development /web-development	published	A. Rakib Hasan	5000	Programming	Intermediate	5/2/2026	Edit Delete
English /english	published	A. Hasan	99	Language	Beginner	4/27/2026	Edit Delete

Figure 7.14: Course CRUD operations

CoachSys < Search...

Students
Search, block/unblock, and delete student accounts. [Refresh](#)

Search name or email... All

STUDENT	STATUS	JOINED	ACTIONS
masud masudparvej@gmail.com	active	5/2/2026	Block Delete
Nazmul nazmul@gmail.com	active	5/2/2026	Block Delete
Masud parvej masud@gmail.com	active	4/26/2026	Block Delete

Figure 7.15: Student List

CoachSys < Search...

Notice Management
Create, pin and delete notices shown in the public notice board. [Refresh](#) [+ Create notice](#)

Search title/content... All types All audiences All

TITLE	TYPE	AUDIENCE	PINNED	PUBLISH	ACTIONS
Final Exam Final Exam starts from 20 May,2026	exam	all	no	5/12/2026	Star Edit Delete
Mid term Exam Mid term exam starts on April 30	exam	all	no	4/27/2026	Star Edit Delete

Figure 7.16: Notice Creation

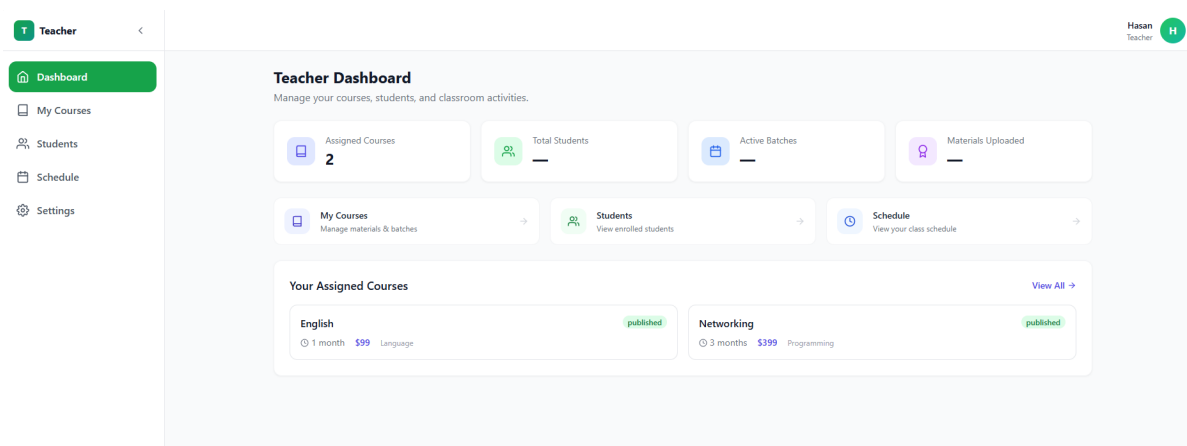


Figure 7.17: Teacher Dashboard

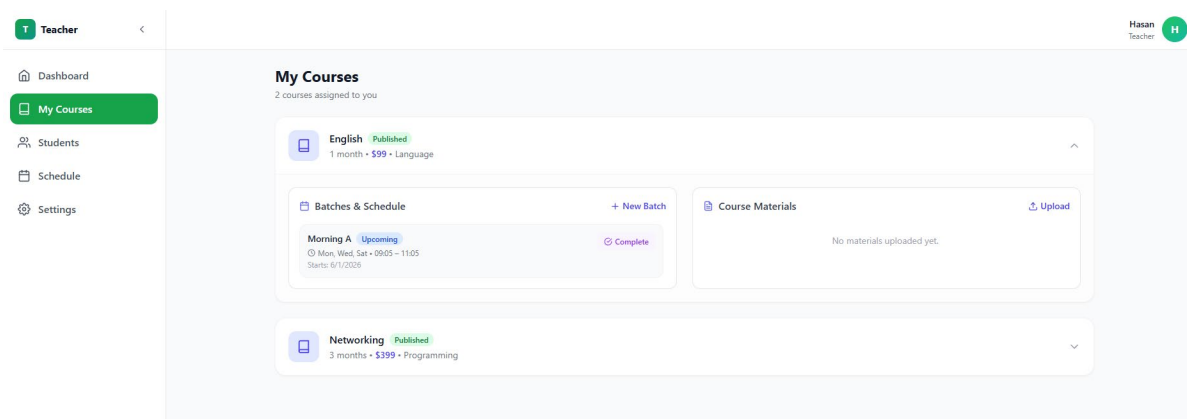


Figure 7.18: Teacher Course list with batch, schedule and material upload

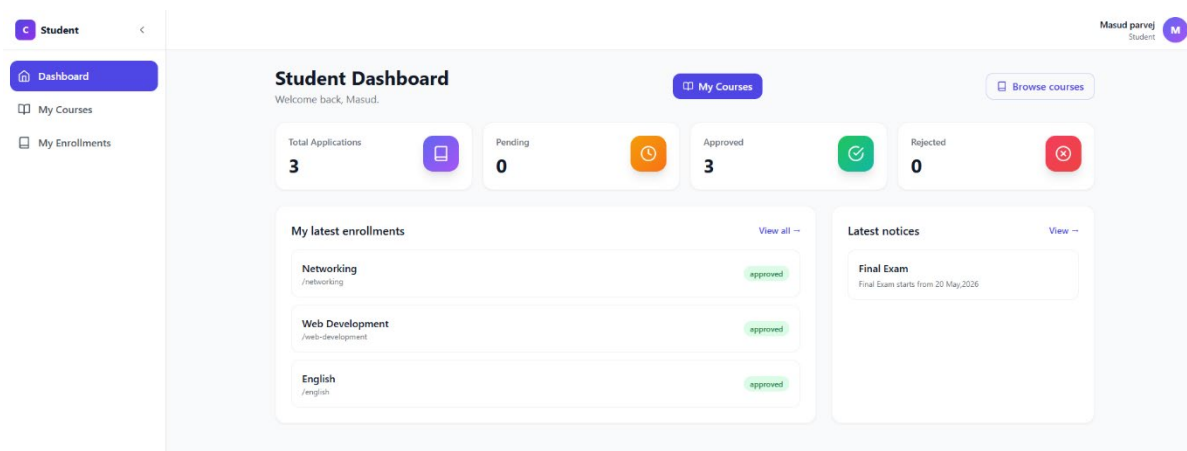


Figure 7.19: Student Dashboard

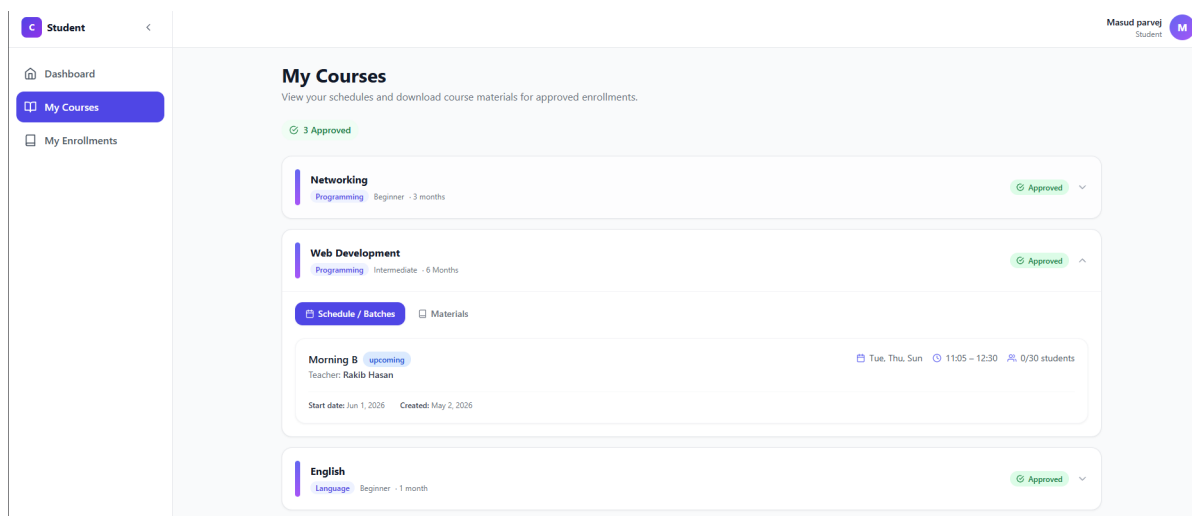


Figure 7.20: Student Course list

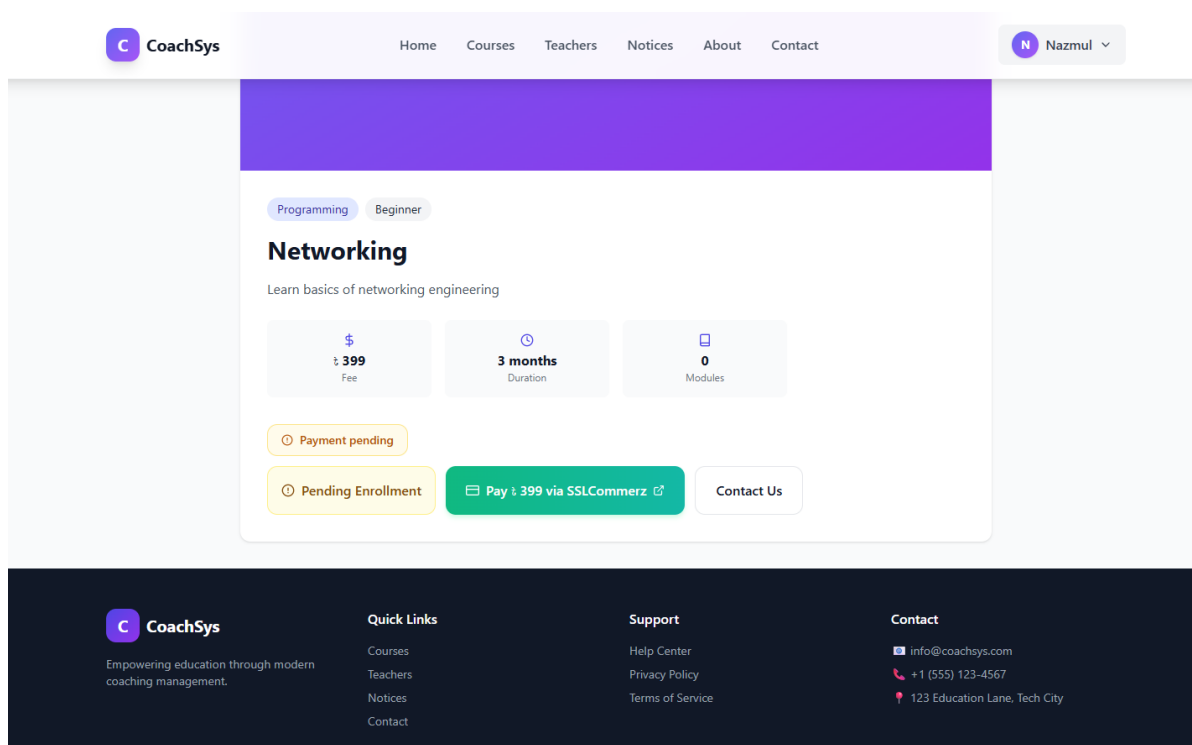


Figure 7.21: Payment system with SSLCOMMERZ

Chapter 8.

System Quality and Testing

8.1 System Quality Management

Software Quality Management (SQM) is a critical procedure that makes sure software satisfies user requirements while keeping laws and quality standards. SQM uses a systematic cycle and rigorous testing to find and address flaws prior to market release. Software Quality Assurance (SQA) is a subset of quality management in software that involves process experts and auditors in addition to testing. SQA keep an eye on every phase of the building process and strives to maintain quality standards in limits of design, safety, reliability, technical and management tests, and maintainability. It guarantees that the procedure and the final software meet high criteria in a number of different areas.

System Quality Management (SQM) for this project wasn't just about finding bugs at the end; it was about integrating quality into every stage of the MERN stack development. By following a structured quality process, I ensured that the code is maintainable, the UI is responsive, and the data remains consistent across all user dashboards.

8.2 Software Quality Management Process

The quality management process followed a three-step cycle to ensure the final product met the initial requirements:

- **Quality Planning:** During the initial 12-week planning phase, I defined specific quality standards, such as ensuring all API endpoints return proper status codes and the Tailwind CSS UI remains functional on mobile devices.
- **Quality Assurance (QA):** Throughout the development of each increment, I applied best practices like modular coding in React and schema validation in MongoDB to prevent errors before they occurred.
- **Quality Control (QC):** This involved the actual testing phase where the system was put through various scenarios to identify any functional gaps or UI glitches that needed fixing before the final report.

8.3 Software Test Cases

8.3.1 System Testing

During the crucial system testing phase of the software development life cycle (SDLC), every component of the integrated system is tested to make sure it satisfies requirements and operates as intended.

Important aspects of system testing consist of:

- **Customer-Centric Review:** This kind of evaluation does not necessitate technical coding or design knowledge; instead, it focuses on user perspectives and experiences to ascertain whether software operates as intended.
- **Operation validation:** Aims to confirm that the software's external elements, such as its behavior and input responses, meet predetermined standards and requirements.
- **Integrated and Integrated Assessing:** System testing carefully examines the software, taking into consideration how various components interact, to ensure seamless integration and proper functionality.

8.3.2 System Testing Methodology

A software product's readiness is usually determined after it has undergone an extensive set of software testing methods at various stages.

Two types of Software Testing:

- White Box Testing
- Black Box Testing

8.3.3 White Box Testing

White box testing, also known as structural or transparent box testing, meticulously examines the internal logic and coding structure of software. It verifies functions by testing branches, paths, statements, and internal logic, ensuring thorough validation of the software's functionalities. Testers require deep coding and logic understanding to identify any issues

within the software. This method, crucial during software implementation, ensures the software performs as expected with provided inputs yielding expected outputs.

8.3.4 Black Box Testing

Without looking into a system's underlying workings, black box testing, sometimes referred to as functional or behavioral testing, evaluates a system's outputs based on inputs provided and its execution. With this method, every software feature is tested methodically, with the results being the only thing that matters. By assessing the program's functionality and behavior, it confirms that it satisfies the criteria and makes sure the user doesn't need to understand how it functions internally. Black box testing places an emphasis on confirming expected behavior against inputs, irrespective of internal architecture expertise, in contrast to white box testing, which explores internal coding details.

8.3.5 Software Test Cases

Table 8.1: Software Test Cases

Test Case ID	Feature	Scenario	Expected Result	Result
TC-01	Authentication	User logs in with valid Student credentials.	Redirected to Student Dashboard with personalized data.	Pass
TC-02	Course Creation	Admin creates a new course and assigns a teacher.	Course appears on the Public Courses page and Teacher's "My Courses."	Pass
TC-03	Enrollment Logic	Student applies for a course without payment.	Admin sees the request with status "Not Paid"; Student sees "Pending."	Pass
TC-04	Admin Approval	Admin approves an enrollment request.	Student status changes from "Pending" to "Approved" in their dashboard.	Pass
TC-05	Material Upload	Teacher uploads a PDF for a specific course batch.	All students enrolled in that batch can view and download the PDF.	Pass
TC-06	Security Check	A Student tries to access the Admin URL directly.	System denies access and redirects the user to their own dashboard.	Pass

TC-07	Notice Board	Admin publishes a new center-wide notice.	Notice immediately appears on the public Notices page for all visitors.	Pass
TC-08	Data Deletion	Admin deletes a student account.	Student can no longer log in, and their data is removed from the list.	Pass

Chapter 9.

Ethical Consideration

9.1 Ethical Considerations in the Software Development Process

The development of the Coaching Center Management System is grounded in the principle that software should be a tool for empowerment rather than a source of risk. As this platform handles the intersection of education and finance, the ethical framework used during development focused on transparency, protection, and professional accountability.

Ethical Considerations in the Software Development Process

- **Data Privacy and Security:** In the digital age, data is a person's most sensitive asset. During the development process, we treated every student's personal information and every teacher's credential as a high-stakes security priority. By implementing JSON Web Tokens (JWT) for session management and integrating SSLCOMMERZ for financial transactions, we ensured that the system does not just function, but protects. We adopted a "Privacy by Design" approach, ensuring that only the necessary amount of data is collected and that it is stored using modern hashing algorithms to prevent unauthorized access.
- **Intellectual Property (IP):** A coaching center thrives on its proprietary knowledge. One of the core ethical drivers was creating a secure "Materials" repository. By restricting access to course content specifically to students who have fulfilled their financial obligations, the system serves as a guardian for the teacher's intellectual property. This prevents the "leakage" of academic resources and ensures that the labor of the educator is respected and compensated.
- **User Impact and Experience:** We recognize that technology can be a barrier if not designed thoughtfully. Ethical development requires us to consider the "User Impact"—ensuring the interface is accessible to those who may not be tech-savvy. We avoided "dark patterns" (manipulative UI designs) that might trick users into making accidental payments. Instead, we prioritized a clear, honest user flow where status updates (like "Pending" or "Approved") are presented clearly to reduce user anxiety.
- **Bias and Fairness in Automation:** Bias can often hide in code, particularly in approval workflows. To combat this, the enrollment approval system is built on objective,

verifiable data. By linking course access directly to payment confirmation via an external gateway, we remove the possibility of human favoritism. This ensures a level playing field for every student, regardless of their background or personal relationship with the center's staff.

- **Responsibility to Stakeholders:** As a developer, I am accountable to the Admin (the business owner), the Teacher (the service provider), and the Student (the client). My responsibility was to ensure that the system remains reliable and that data integrity is maintained across all three dashboards. If the system crashes, a student might miss a lesson or an admin might lose a financial record; therefore, rigorous testing was an ethical necessity to ensure the system's reliability.
- **Professional Responsibility:** Ethical software engineering requires a commitment to quality. During this project, I maintained a professional standard by documenting code and following industry best practices (the MERN stack architecture). Cutting corners to meet a deadline is an ethical failure; instead, I prioritized a "Minimum Viable Product" (MVP) that is stable and secure over a feature-heavy system that is prone to bugs.

9.2 Sustainability in the Software Development Process

Sustainability in software engineering is the practice of designing systems that are long-lived, resource-efficient, and socially beneficial.

- **Economic Sustainability:** From a business perspective, the system is designed to minimize the "Total Cost of Ownership" (TCO). By using open-source technologies (MongoDB, Express, React, Node.js), the coaching center avoids expensive licensing fees. Furthermore, the modular architecture ensures that the system is scalable. As the center expands its student base, the software can grow with it without requiring a massive reinvestment or a total system overhaul, ensuring long-term financial viability.
- **Social Sustainability:** This project contributes to social sustainability by democratizing access to organized education. By providing a digital hub for notices and materials, we reduce the center's reliance on physical paper and travel, while

fostering a stronger, more connected academic community. The system ensures that information is distributed equitably to all students simultaneously, regardless of their physical location, which is a vital step toward sustainable and inclusive education.

Chapter 10.

Conclusion

10.1 Brief Overview of the Project

The Coaching Center Management System was born out of a need to modernize the administrative and academic workflows of private education centers. Developed using the MERN stack, the project successfully integrated three specialized dashboards that manage the entire lifecycle of a student from public browsing and enrollment to payment, material consumption, and assignment submission. By centralizing these disparate tasks into one cohesive platform, the project has replaced inefficient manual tracking with a high-performance digital ecosystem.

10.2 Proposed System Benefits

The implementation of this system offers transformative benefits for all stakeholders. For the Administration, it provides a single "source of truth" for financial and academic data, drastically reducing the time spent on manual record-keeping. Teachers benefit from a streamlined classroom management tool that allows them to focus on pedagogy rather than paperwork. Finally, Students gain a transparent and structured learning environment where their progress is tracked, their materials are always accessible, and their payments are handled through secure, industry-standard gateways like SSLCOMMERZ.

10.3 Limitations of the Project

Despite the robust feature set, certain limitations remain that define the scope for future iterations. Currently, the system functions as an Asynchronous Learning Platform; it handles file sharing and assignments well but does not yet support synchronous, real-time video lecturing. Additionally, the lack of an automated "forget password" email recovery system and a native mobile application (for offline study) are areas where the current web-based architecture could be further enhanced to meet the needs of users in low-connectivity environments.

10.4 Practicum and Its Value

This practicum served as the ultimate test of my academic journey. It required the integration of multiple disciplines: database management, frontend design, backend logic, and cybersecurity. The value of this experience lies in the transition from "writing code" to "solving problems." I learned to navigate the complexities of third-party API integration (SSLCOMMERZ) and the critical importance of Role-Based Access Control (RBAC). This project has not only sharpened my technical skills but also developed my ability to manage a

full software development lifecycle (SDLC) from requirements gathering to final deployment.

10.5 Future Plan

Moving forward, the roadmap for this project is ambitious. The next phase will involve integrating technology to support live, in-app video conferencing, turning the platform into a true "Virtual Classroom." I also intend to implement Automated Push Notifications via a dedicated mobile app to keep students updated on deadlines in real-time. Finally, the inclusion of an AI-driven Analytics Dashboard will eventually allow admins to predict student performance trends and optimize course offerings based on historical data.

References

Chodorow, K. (2013) *MongoDB: The Definitive Guide*. 2nd edn. Sebastopol: O'Reilly Media.

Flanagan, D. (2020) *JavaScript: The Definitive Guide*. 7th edn. Sebastopol: O'Reilly Media.

Haverbeke, M. (2018) *Eloquent JavaScript: A Modern Introduction to Programming*. 3rd edn. San Francisco: No Starch Press.

MongoDB (2026) *The Document Data Model*. Available at: <https://www.mongodb.com/docs/>.

React (2026) *React Documentation: Managing State*. Available at: <https://react.dev/>.

Sommerville, I. (2016) *Software Engineering*. 10th edn. Pearson Education.

Tailwind Labs (2024) *Tailwind CSS Documentation*. Available at: <https://tailwindcss.com/docs>.

SSLCOMMERZ (2026) *Developer Documentation: Integration Guide*. Available at: <https://developer.sslcommerz.com/docs/>.